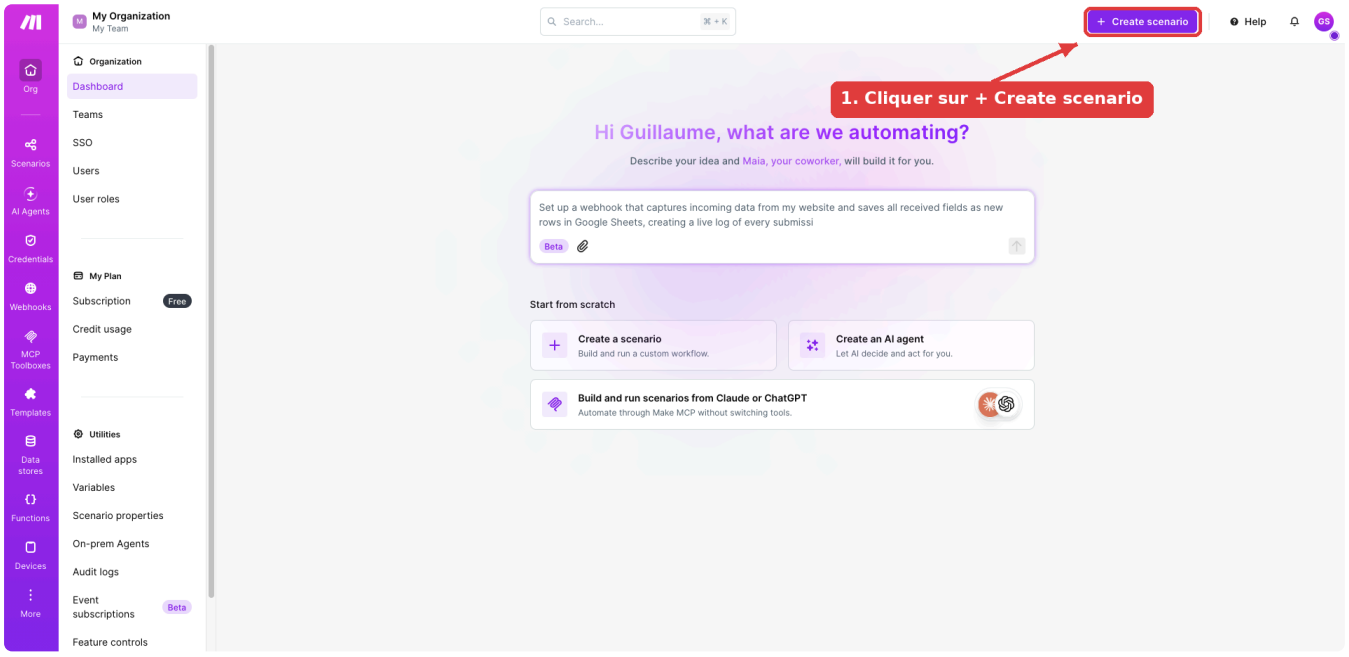
Tuto — Construire la veille RSS dans Make

Nov 8, 2026 · @Guillaume SOULAT

Ce tuto construit un scénario Make qui lit plusieurs flux RSS chaque semaine, garde les articles récents qui parlent de data et les écrit dans Google Sheets. Prérequis : la connexion Google Sheets du tuto précédent et un Sheet avec les 6 en-têtes.

Étape 1 — Créer un nouveau scénario

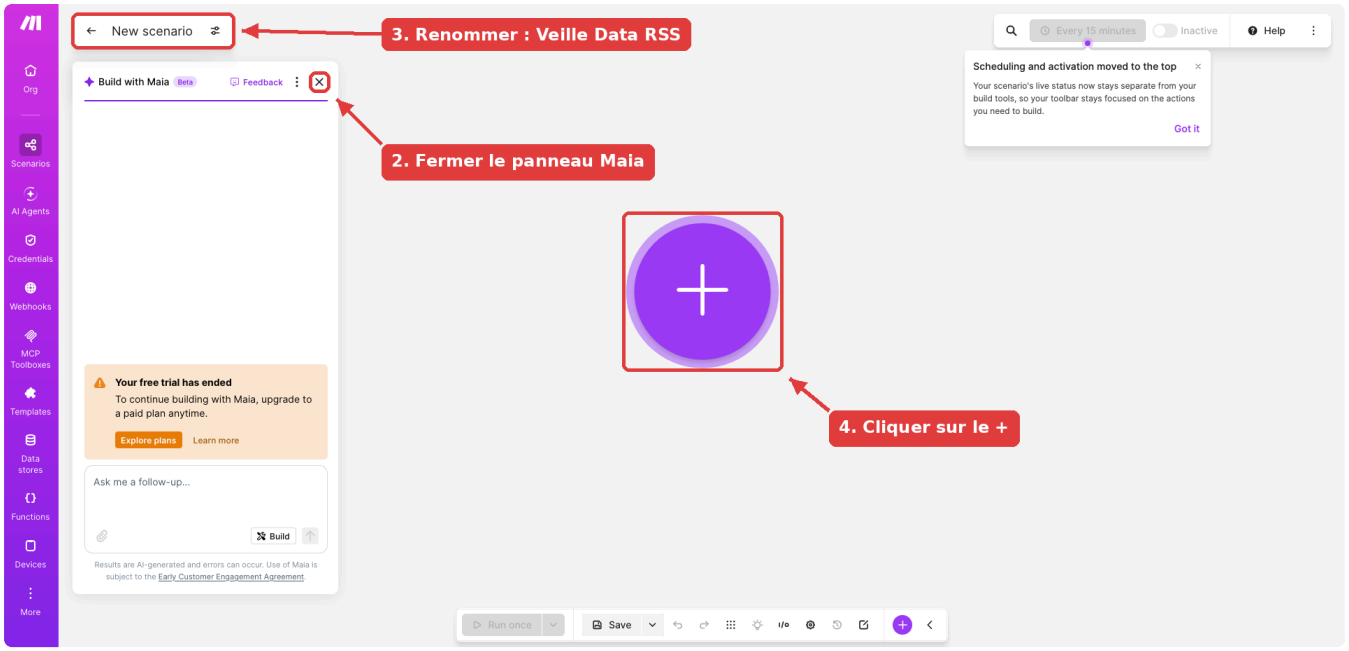
Depuis l'accueil de Make, cliquez sur le bouton violet **+ Create scenario** en haut à droite. La carte **Create a scenario**, au centre de la page, fait la même chose.



N'utilisez pas le champ de texte « Describe your idea » : il fait construire le scénario par l'IA de Make, alors que le but est de le construire vous-même.

L'éditeur de scénario s'ouvre. Avant d'ajouter un module :

1. Fermez le panneau **Build with Maia** à gauche avec la croix. Sur l'offre gratuite, il affiche « Your free trial has ended » : c'est normal, on n'en a pas besoin.
2. Si une bulle « Scheduling and activation moved to the top » apparaît en haut à droite, cliquez sur **Got it**.
3. Cliquez sur **New scenario** en haut à gauche et renommez le scénario `Veille Data RSS`.
4. Cliquez sur le grand **+** violet au centre pour ajouter le premier module.



> **Sauvegardez souvent.** Le bouton **Save** de chaque fenêtre de module ferme la fenêtre, mais n'enregistre pas le scénario. Pour enregistrer le scénario, cliquez sur **Save** dans la barre du bas : tant qu'un petit point orange apparaît sur son icône, des modifications ne sont pas enregistrées. Prenez le réflexe après chaque étape : si l'onglet se ferme ou si la connexion coupe, vous ne perdez rien.

Pourquoi ce montage

Un scénario Make ne peut avoir qu'un seul déclencheur, et le déclencheur RSS (Watch RSS feed items) ne surveille qu'un seul flux. Pour suivre plusieurs sources, on remplace ce déclencheur par une chaîne de 4 modules :

1. **Iterator** : il contient la liste des URL de flux et les envoie une par une au module suivant. Pourquoi : chaque flux est traité de la même façon, on écrit les réglages une seule fois.
2. **RSS > Retrieve RSS feed items** : il lit les derniers articles de chaque flux. Pourquoi : contrairement à Watch, il accepte une URL différente à chaque passage.
3. **Filtre** sur les mots-clés. Pourquoi : la plupart des flux sont généralistes ou en anglais, le filtre ne garde que les articles qui parlent vraiment de data. La date, elle, est gérée directement dans le module RSS (champ Date from) : sans elle, Retrieve relirait tout le flux et les mêmes articles reviendraient à chaque exécution en doublon.
4. **Google Sheets > Add a Row** : il range chaque article retenu dans le tableau.

Le scénario tourne une fois par semaine, le lundi matin, juste avant le point veille de l'équipe. Chaque exécution coûte 1 opération pour l'Iterator, 1 par flux pour le module RSS (8), et 1 par article retenu pour Google Sheets. Le filtre, lui, ne coûte rien. Avec 4 à 5

exécutions par mois, on reste autour de 100 à 200 opérations par mois, très en dessous de la limite de 1 000 de l'offre gratuite.

Les 8 sources de la veille

Ces 8 flux ont tous répondu avec des articles de novembre 2026, et la formule de l'Iterator les contient tous. La plupart des flux sont en anglais, d'où les mots-clés dans les deux langues à l'étape 6.

Source	Langue	Ce qu'on y trouve	URL du flux
Google News (flux du brief)	FR	Actualité data, données, Power BI	`https://news.google.com/rss/search?q=%22data%22+OR+%22donn%C3%A9es%22+OR+%22Power+BI%22&hl=fr&gl=FR&ceid=FR:fr`
Le Big Data	FR	Actualité data et IA, très orientée IA	`https://www.lebigdata.fr/feed`
Towards Data Science	EN	Tutoriels analyse, statistiques, KPI, Python	`https://towardsdatascience.com/feed`
KDnuggets	EN	Outils, librairies Python, SQL, formations	`https://www.kdnuggets.com/feed`
SQLBI	EN	Power BI et DAX, par des références du sujet	`https://www.sqlbi.com/feed/`
FlowingData	EN	Datavisualisation, exemples commentés	`https://flowingdata.com/feed`
Storytelling with Data	EN	Présenter des données, graphiques, slides	`https://www.storytellingwithdata.com/blog?format=rss`
Analytics Vidhya	EN	Tutoriels data et IA, niveau débutant	`https://www.analyticsvidhya.com/feed/`

C'est cette liste que l'Iterator va parcourir à l'étape suivante : les 8 URL y seront collées dans une seule formule, séparées par des virgules.

Étape 2 — Lister les flux avec un Iterator

Pourquoi commencer par un Iterator : c'est lui qui « tient » la liste des flux. À chaque exécution, il envoie la première URL au module RSS, puis la deuxième, puis la troisième. Ajouter ou retirer une source se fera donc en modifiant une seule ligne, sans toucher au reste du scénario.

Si vous connaissez Python

Make utilise les mêmes idées que Python, avec d'autres mots :

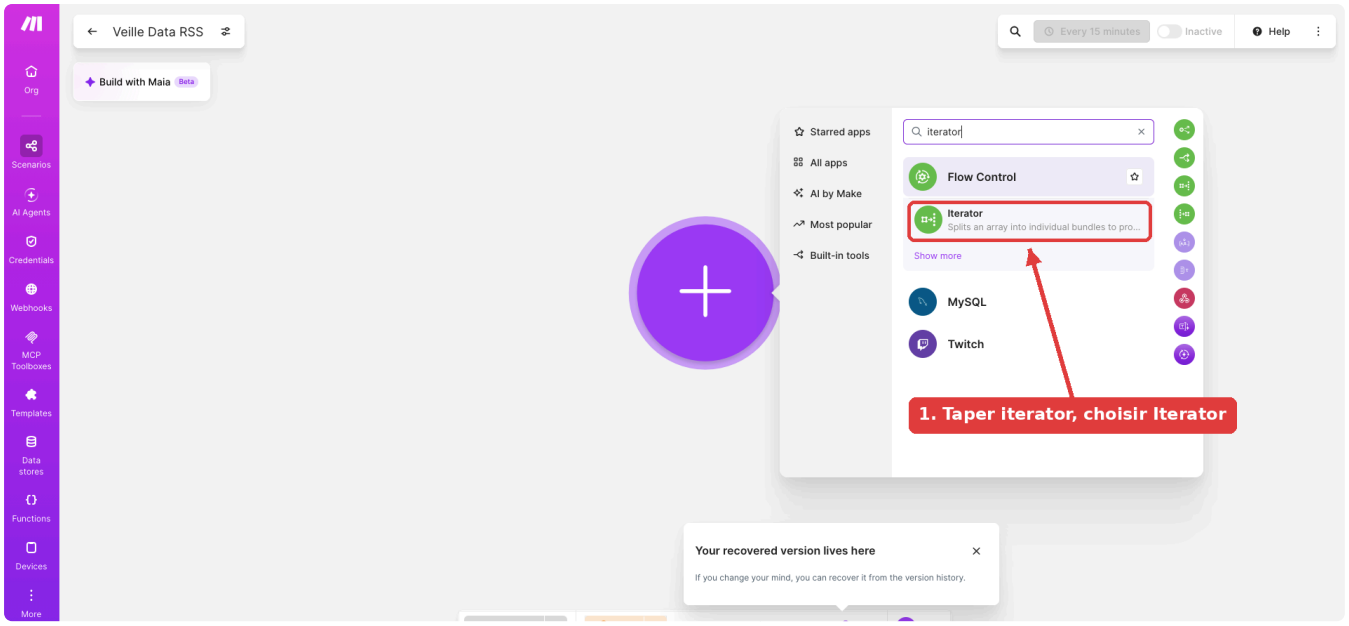
Dans Make	En Python	Ce que c'est
Array (tableau)	`list`	Une suite de valeurs dans un ordre : ici, les 8 URL de flux
`split(texte; ",")`	`texte.split(",")`	Coupe un texte à chaque virgule et renvoie une liste
Iterator	une boucle `for`	Prend la liste et passe ses éléments un par un à la suite
Bundle (paquet)	l'élément courant de la boucle	Ce qui circule entre deux modules : ici, une URL
Les modules après l'Iterator	le corps de la boucle	Ce qui est exécuté pour chaque élément

Le scénario que vous construisez fait donc l'équivalent de ce code :

```
flux = "url_google_news,url_tds,url_sqlbi".split(",")   # l'Array de
l'Iterator

for url in flux:                                          # l'Iterator
    articles = lire_flux_rss(url, depuis=il_y_a_7_jours, maximum=20) # RSS >
Retrieve RSS feed items
    for article in articles:
        if parle_de_data(article):                        # le filtre mots-
clés
            ajouter_ligne(sheet, article)                 # Google Sheets >
Add a Row
```

1. Cliquez sur le grand **+** au centre de l'éditeur.
2. Tapez `iterator` dans la recherche. Make affiche l'app **Flow Control** et, dessous, le module **Iterator**.
3. Cliquez sur **Iterator**.

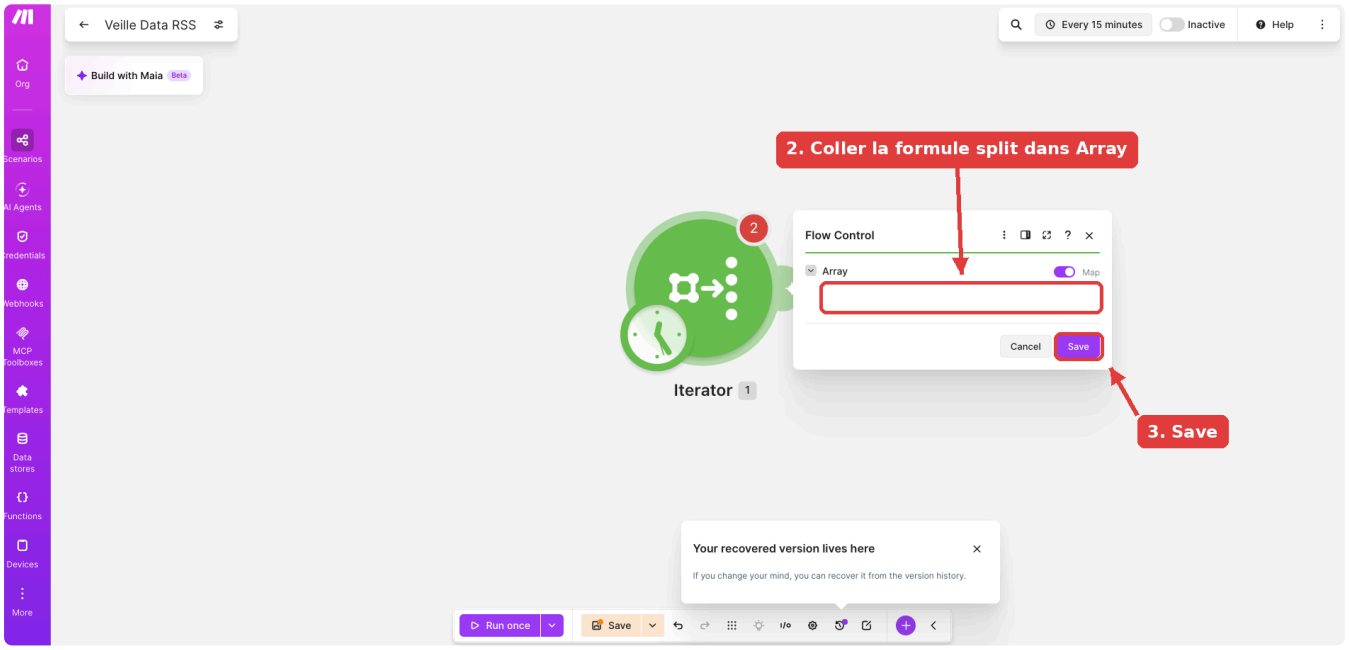


Le module Iterator est posé et sa fenêtre **Flow Control** s'ouvre avec un seul champ, **Array**. C'est là qu'on donne la liste des flux.

1. Laissez l'interrupteur **Map** activé : il permet de taper une formule au lieu de choisir une valeur dans une liste.
2. Collez dans **Array** la formule suivante (une seule ligne) :

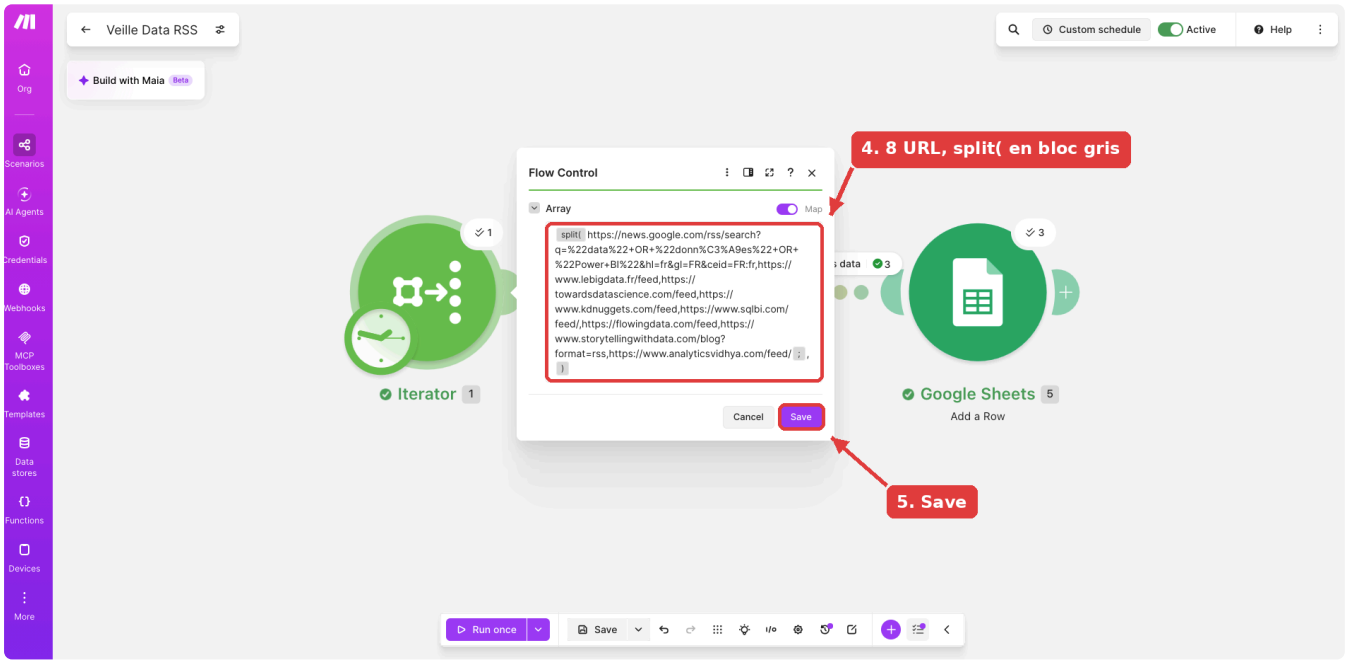
```
{{split("https://news.google.com/rss/search?q=%22data%22+OR+%22donn%C3%A9es%22+OR+%22Power+BI%22&hl=fr&gl=FR&ceid=FR:fr,https://www.lebigdata.fr/feed,https://towardsdatascience.com/feed,https://www.kdnuggets.com/feed,https://www.sqlbi.com/feed/,https://flowingdata.com/feed,https://www.storytellingwithdata.com/blog?format=rss,https://www.analyticsvidhya.com/feed/"; ",")}}
```

3. Cliquez sur **Save**.



Pourquoi `split` : le champ Array attend une liste, pas un texte. `split(texte; ",")` coupe le texte à chaque virgule, exactement comme `texte.split(",")` en Python, et renvoie une liste de 8 URL, une par flux du tableau ci-dessous. Pour ajouter une source, ajoutez une virgule puis son URL avant le guillemet fermant ; pour en retirer une, supprimez son URL et la virgule qui la précède. Jamais d'espace autour des virgules : il ferait partie de l'URL et le flux ne répondrait pas.

Vérifier que Make a compris la formule. Après le collage, `split(` , le `;` et la `)` deviennent des petits blocs gris, et les guillemets disparaissent : les blocs gris montrent que Make a reconnu la fonction. Collez toujours une formule en entier, d'un seul coup, accolades comprises.



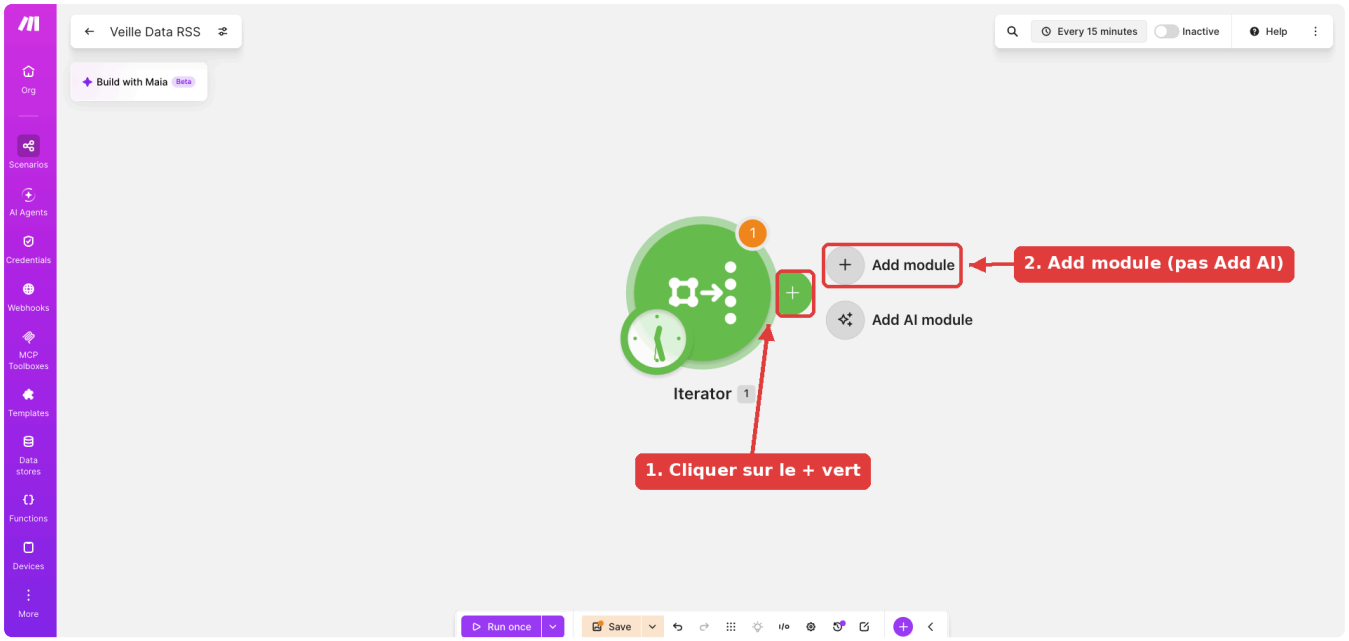
Un panneau de fonctions s'ouvre aussi à gauche, avec un encart « Unlock functions on Enterprise ». Ignorez-le : `split` fait partie des fonctions gratuites.

Si une bulle « Your recovered version lives here » apparaît en bas, fermez-la avec la croix : elle rappelle seulement que les anciennes versions du scénario restent récupérables.

Sauvegardez : cliquez sur **Save** dans la barre du bas.

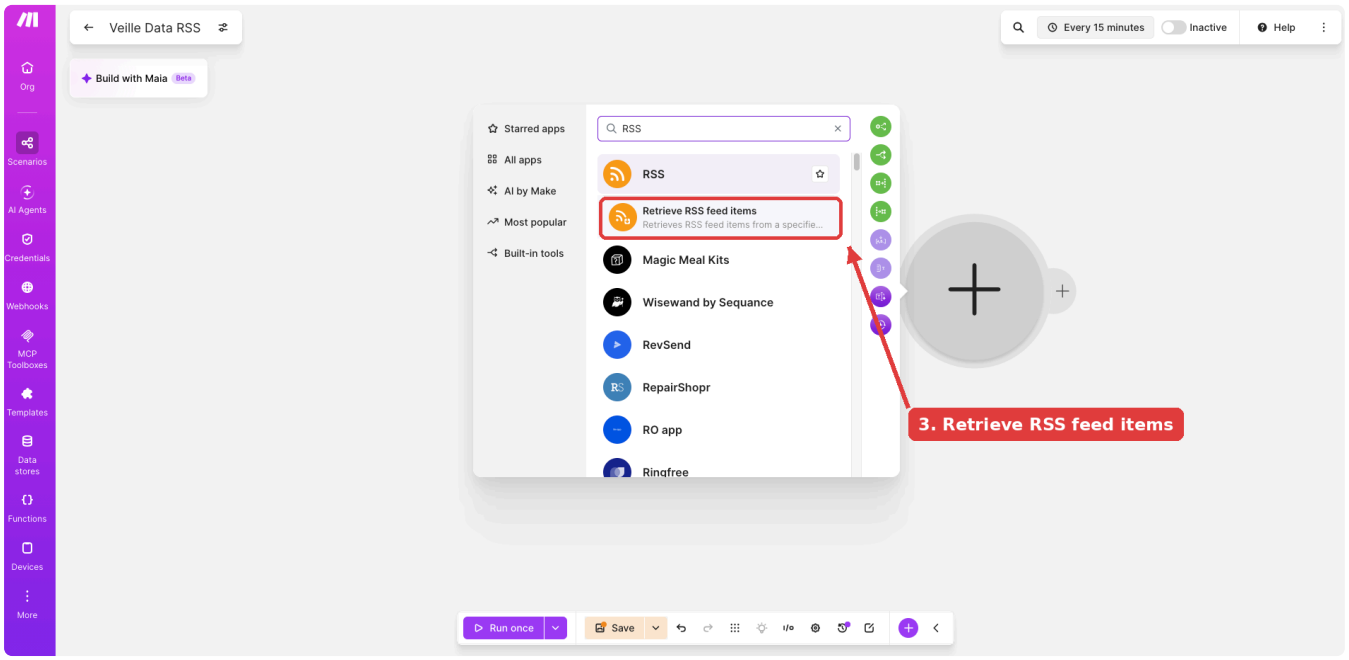
Étape 3 — Ajouter le module RSS qui lit chaque flux

Une fois l'Iterator enregistré, un **+** vert apparaît sur son bord droit. Cliquez dessus : un petit menu propose **Add module** et **Add AI module**. Choisissez **Add module**. Pourquoi pas Add AI module : ce brief construit une veille sans IA, les articles sont triés par des règles (date, mots-clés) que vous écrivez vous-même.



Dans la liste des applications qui s'ouvre, RSS n'apparaît pas tout de suite : tapez `RSS` dans le champ **Search all apps or modules**.

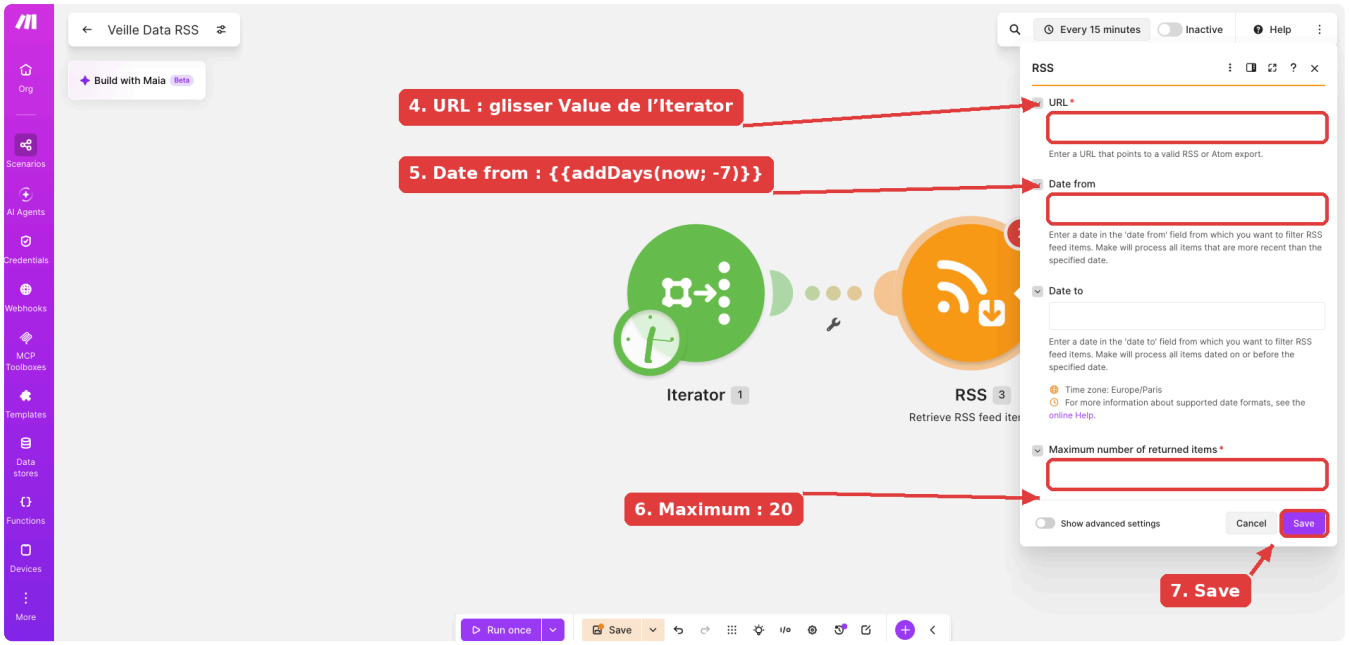
Sous l'app **RSS**, un seul module est proposé : **Retrieve RSS feed items**. Cliquez dessus.



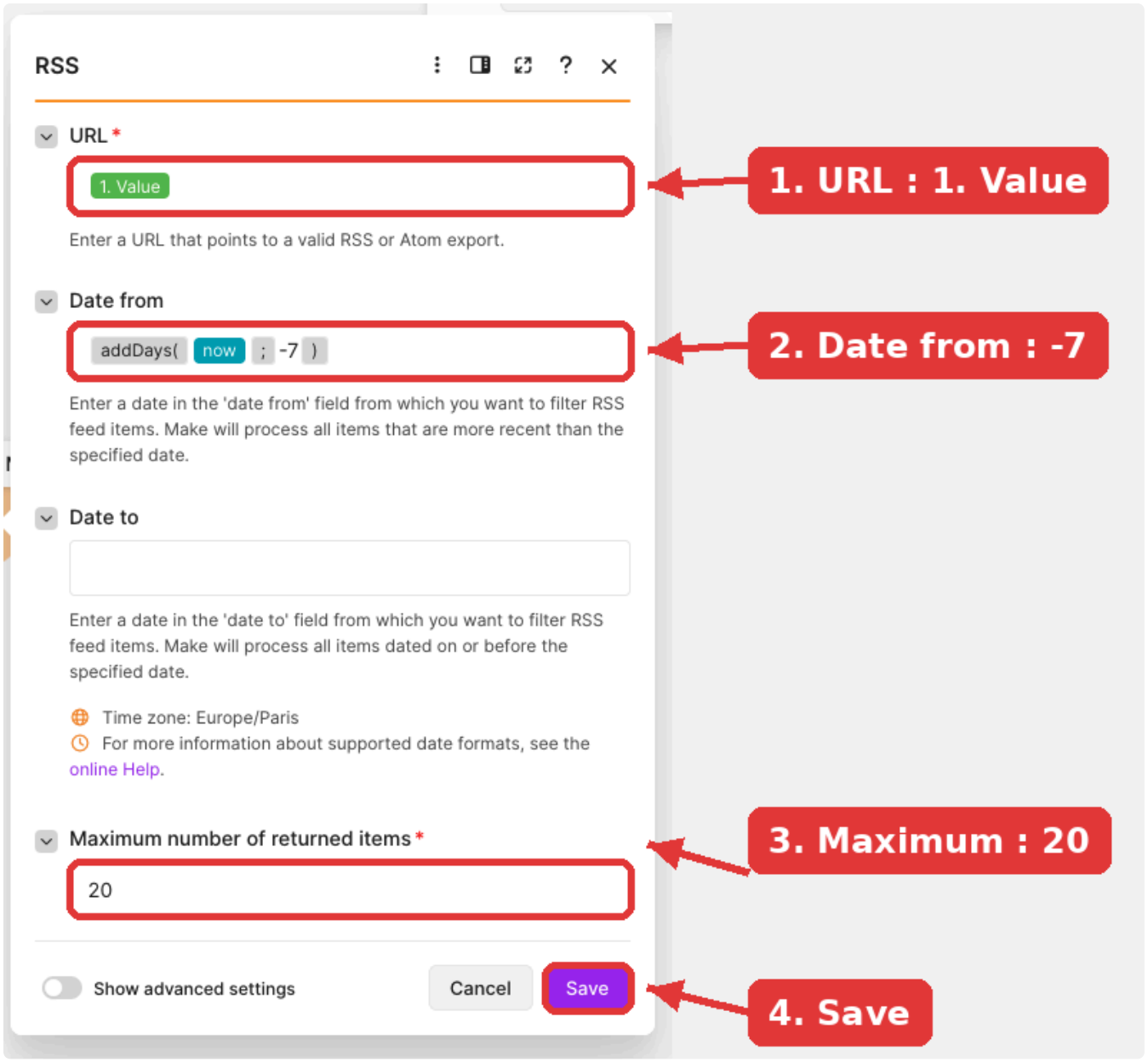
Pourquoi Watch RSS feed items a disparu : c'est un déclencheur, et un déclencheur ne peut être que le premier module d'un scénario. Ici, la place est déjà prise par l'Iterator, donc Make ne propose que les modules utilisables à cet endroit. En Python, c'est la différence entre le point d'entrée du programme et une fonction appelée dans la boucle.

La fenêtre de réglages du module RSS s'ouvre à droite. Le module est déjà relié à l'Iterator par les petits points entre les deux.

1. **URL** : cliquez dans le champ. Un panneau s'ouvre avec les données des modules précédents. Dans la partie **Iterator**, cliquez sur **Value** (ou glissez-la dans le champ). Pourquoi : on n'écrit pas une URL fixe, on prend celle que l'Iterator envoie à ce passage. C'est le `url` de la boucle `for url in flux`.
2. **Date from** : collez `{{addDays(now; -7)}}`. Pourquoi : le scénario tournera une fois par semaine, donc le module doit lire les articles des 7 derniers jours. Avec -1, on ne lirait que la veille et on perdrait les 6 autres jours ; avec plus de 7, les articles de la semaine précédente reviendraient en doublon.
3. **Date to** : laissez vide.
4. **Maximum number of returned items** : `20`. Pourquoi : une semaine d'articles dépasse largement 5 par flux. Le module RSS compte 1 opération par flux, quel que soit le nombre d'articles lus : monter à 20 ne coûte donc rien de plus. Seuls les articles retenus par le filtre coûtent une opération dans Google Sheets.
5. Cliquez sur **Save**.



En Python, ce module équivaut à `articles = lire_flux_rss(url, depuis=il_y_a_7_jours, maximum=20)`.

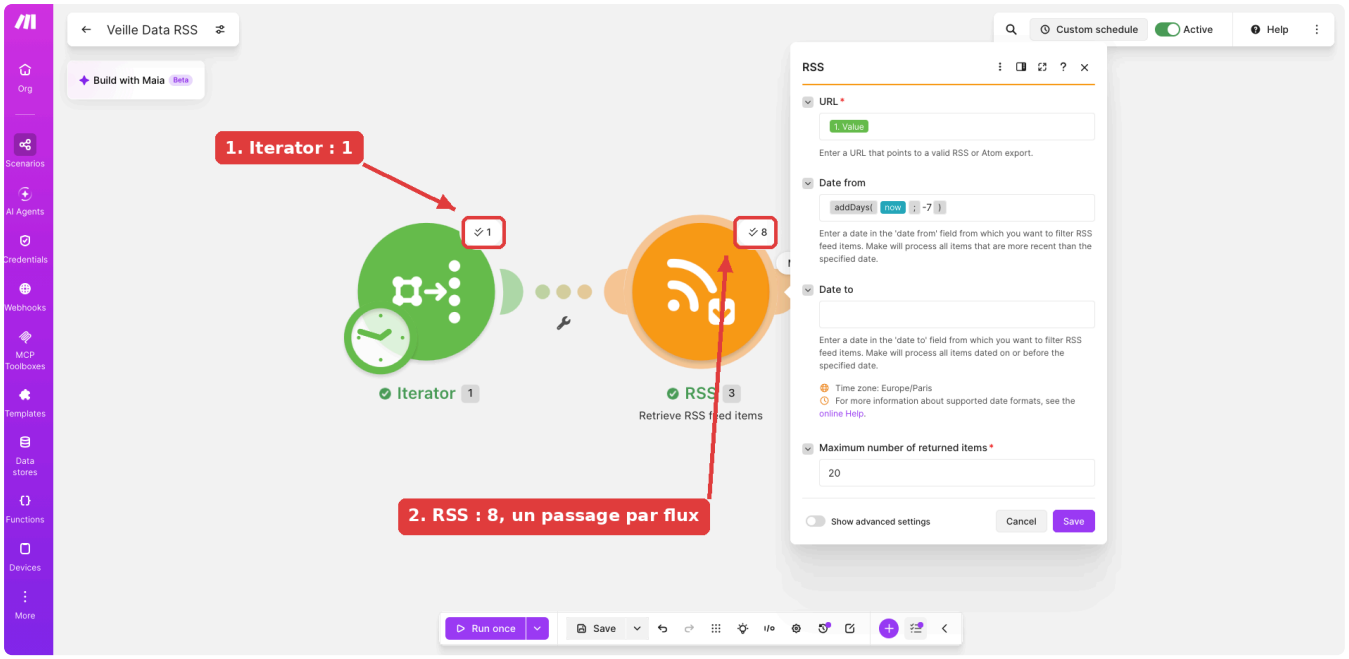


Sauvegardez : cliquez sur **Save** dans la barre du bas.

Étape 4 — Tester avec Run once

Pourquoi tester maintenant, avant Google Sheets : on vérifie que les flux répondent et on laisse Make découvrir les champs des articles. Sans ce test, ces champs ne seraient pas proposés au moment de remplir le Sheet.

1. Cliquez sur **Run once** en bas à gauche.
2. Après quelques secondes, chaque module affiche une coche verte et une bulle avec un chiffre. Cliquez sur la bulle du module RSS.



Lire les bulles. Le chiffre est le nombre de fois où le module a tourné :

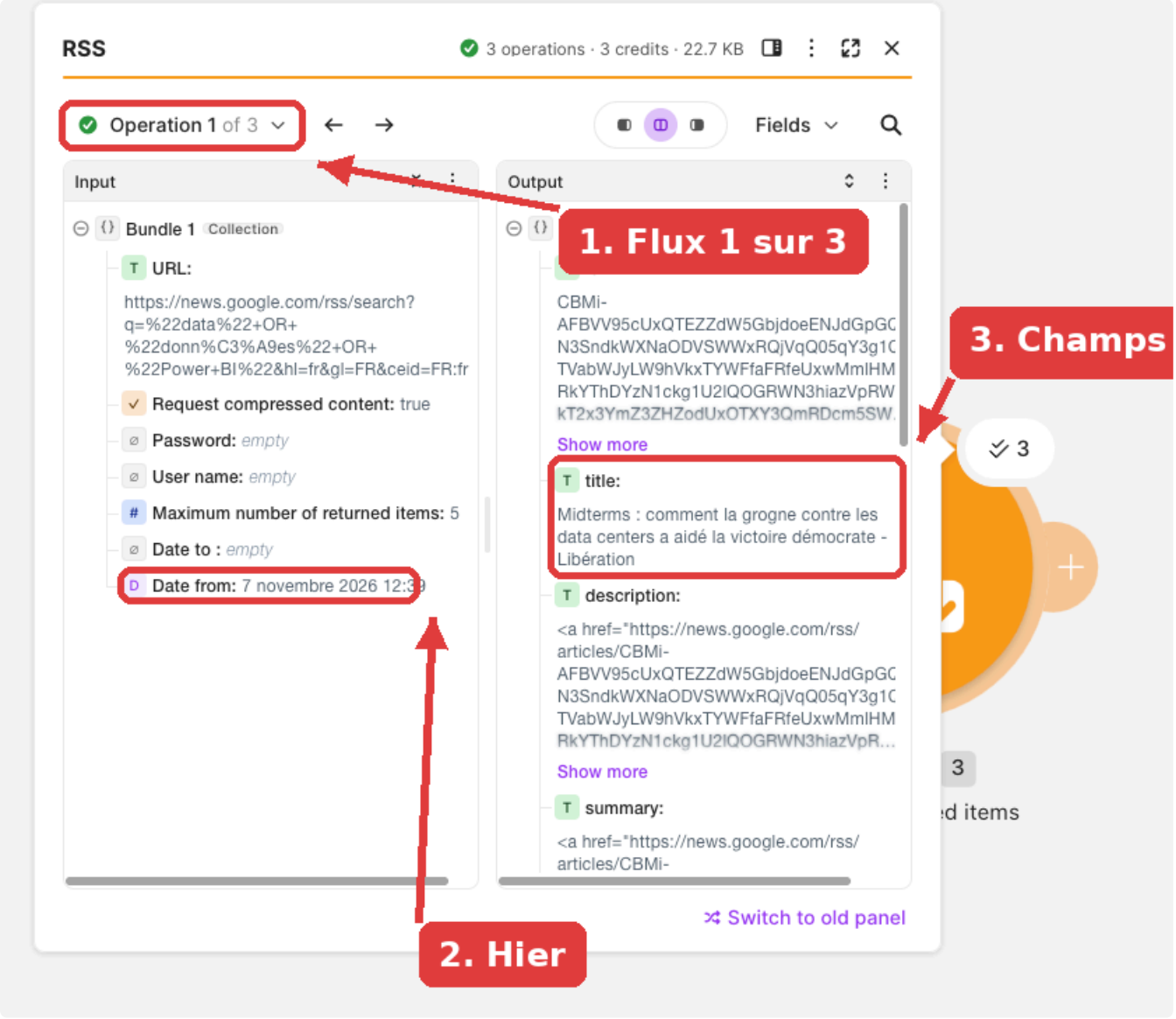
Module	Bulle	Ce que ça veut dire	En Python
Iterator	1	Il a tourné une fois et a sorti la liste des 8 URL	`flux = [...]`
RSS	8	Il a tourné 8 fois, une fois par flux	les 8 tours de `for url in flux`

Lire le détail du module RSS. La fenêtre qui s'ouvre a deux colonnes :

- **Operation 1 of …** en haut : le premier passage du module, donc le premier flux. Les flèches ← → à côté permettent de voir les autres flux, un par passage.
- **Input** (à gauche) : ce que le module a reçu. On y retrouve l'URL envoyée par l'Iterator, et **Date from** calculée par la formule. Sur cette capture, prise lors d'un premier test à -1, c'est la date d'hier ; avec le réglage hebdomadaire, vous verrez la date d'il y a 7 jours. C'est la preuve que la formule `addDays` fonctionne.
- **Output** (à droite) : ce que le module renvoie. Chaque **Bundle** est un article, avec ses champs : `title` (le titre), `description` et `summary` (le résumé), et plus bas `url` (le lien) et `dateCreated` (la date de publication). Ce sont ces champs qu'on rangera dans les colonnes du Sheet à l'étape 5.

En Python, chaque bundle serait un dictionnaire : `article["title"]`, `article["url"]`, `article["dateCreated"]`.

Remarque : le champ `description` de Google News contient du HTML (des balises `<a href=...>`). Il faudra le nettoyer avec `stripHTML` avant de l'écrire dans le Sheet.



Étape 5 — Écrire chaque article dans Google Sheets

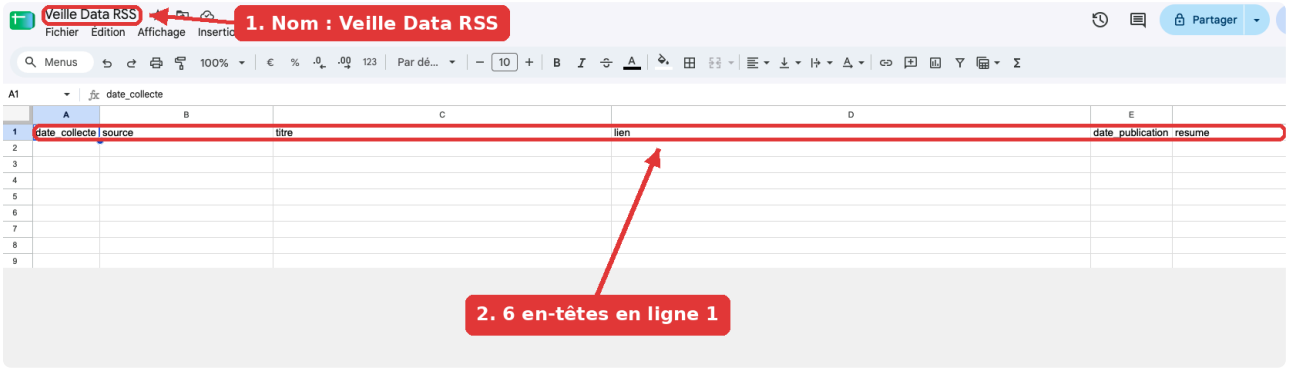
Pourquoi ce module : jusqu'ici les articles n'existent que pendant l'exécution, comme une variable Python qui disparaît à la fin du programme. **Add a Row** les enregistre dans le Sheet, où l'équipe pourra les lire, les trier et les compter.

Avant de commencer cette étape, deux prérequis :

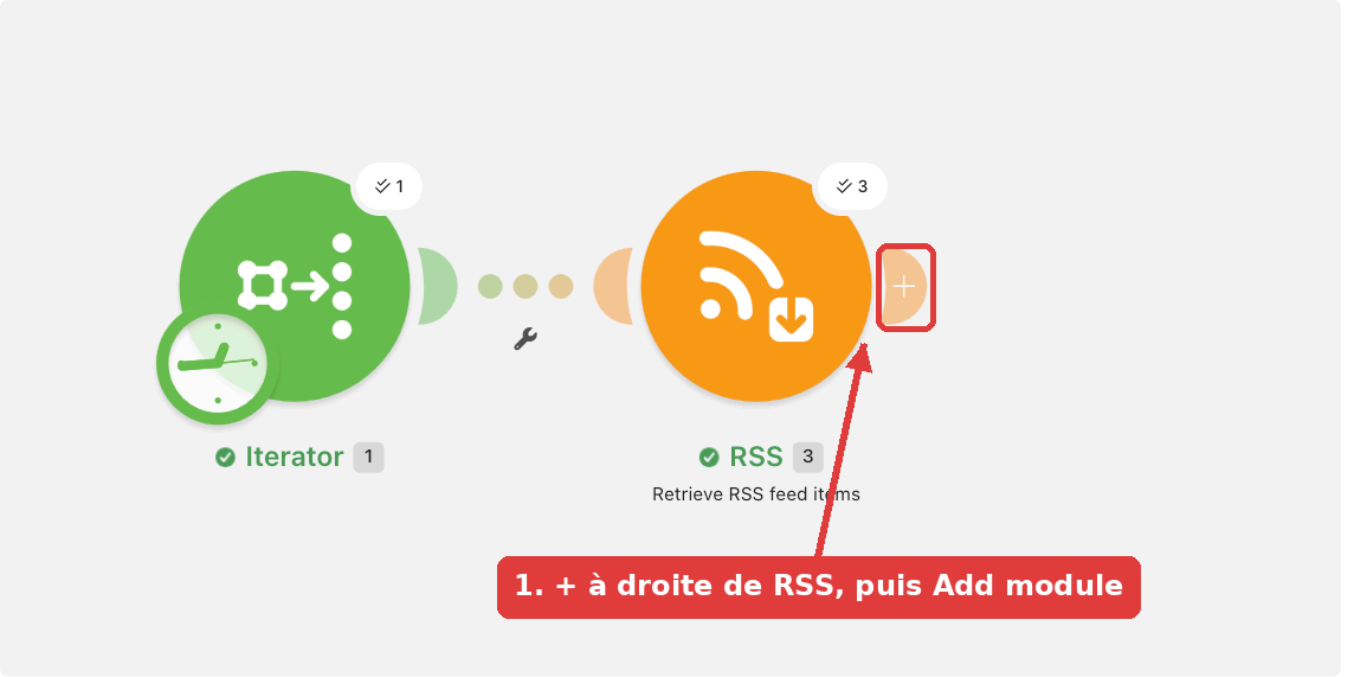
1. **La connexion Google Sheets doit exister dans Make.** Si ce n'est pas fait, suivez d'abord le tuto Connecter Google Sheets à Make (Credentials > Connections > + Connection). Pourquoi : Make a besoin de votre autorisation Google pour écrire dans vos fichiers. Sans connexion, le champ Connection reste vide et le module ne peut rien enregistrer.
2. **Un Google Sheets nommé `Veille Data RSS` doit exister dans votre Drive**, avec ses 6 en-têtes en ligne 1. Pour le créer :
   - Ouvrez drive.google.com avec le compte Google utilisé pour la connexion Make.

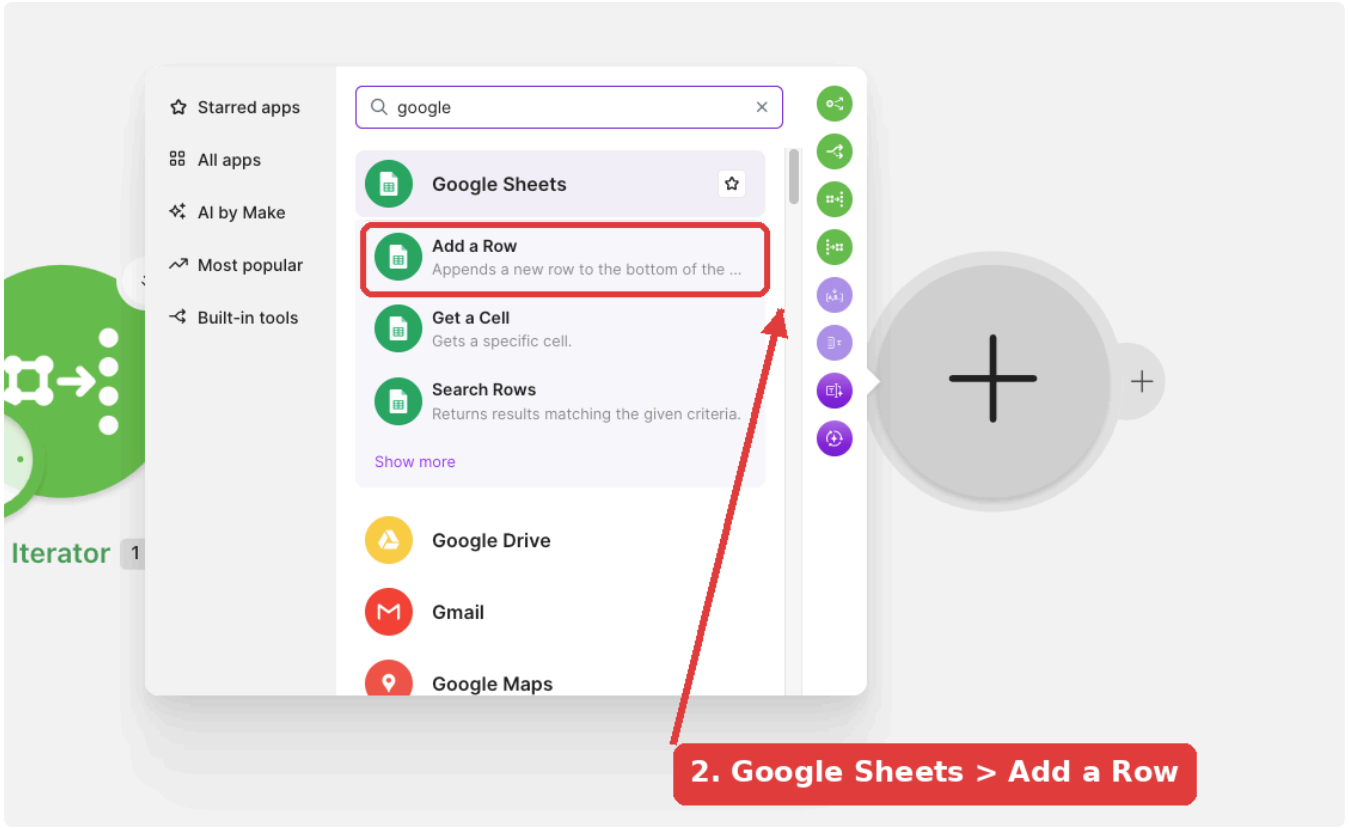
- Cliquez sur **+ Nouveau** > **Google Sheets**.
- Cliquez sur « Feuille de calcul sans titre » en haut à gauche et renommez le fichier `Veille Data RSS`.
- Tapez les en-têtes en ligne 1, une par cellule de A1 à F1 : `date_collecte`, `source`, `titre`, `lien`, `date_publication`, `resume`.

Pourquoi : Make ne crée pas le fichier tout seul, il écrit dans un Sheet existant. Il lit ces en-têtes pour afficher un champ par colonne à remplir, et le nom `Veille Data RSS` permet de le retrouver facilement. Utilisez le même compte Google que la connexion, sinon le fichier n'apparaîtra pas dans Make.
Profitez-en pour mettre la colonne E au bon format : cliquez sur la lettre **E**, puis **Format** > **Nombre** > **Date**. Pourquoi : Google Sheets stocke les dates comme des nombres ; le format Date les affiche lisiblement (08/11/2026 au lieu de 46334).



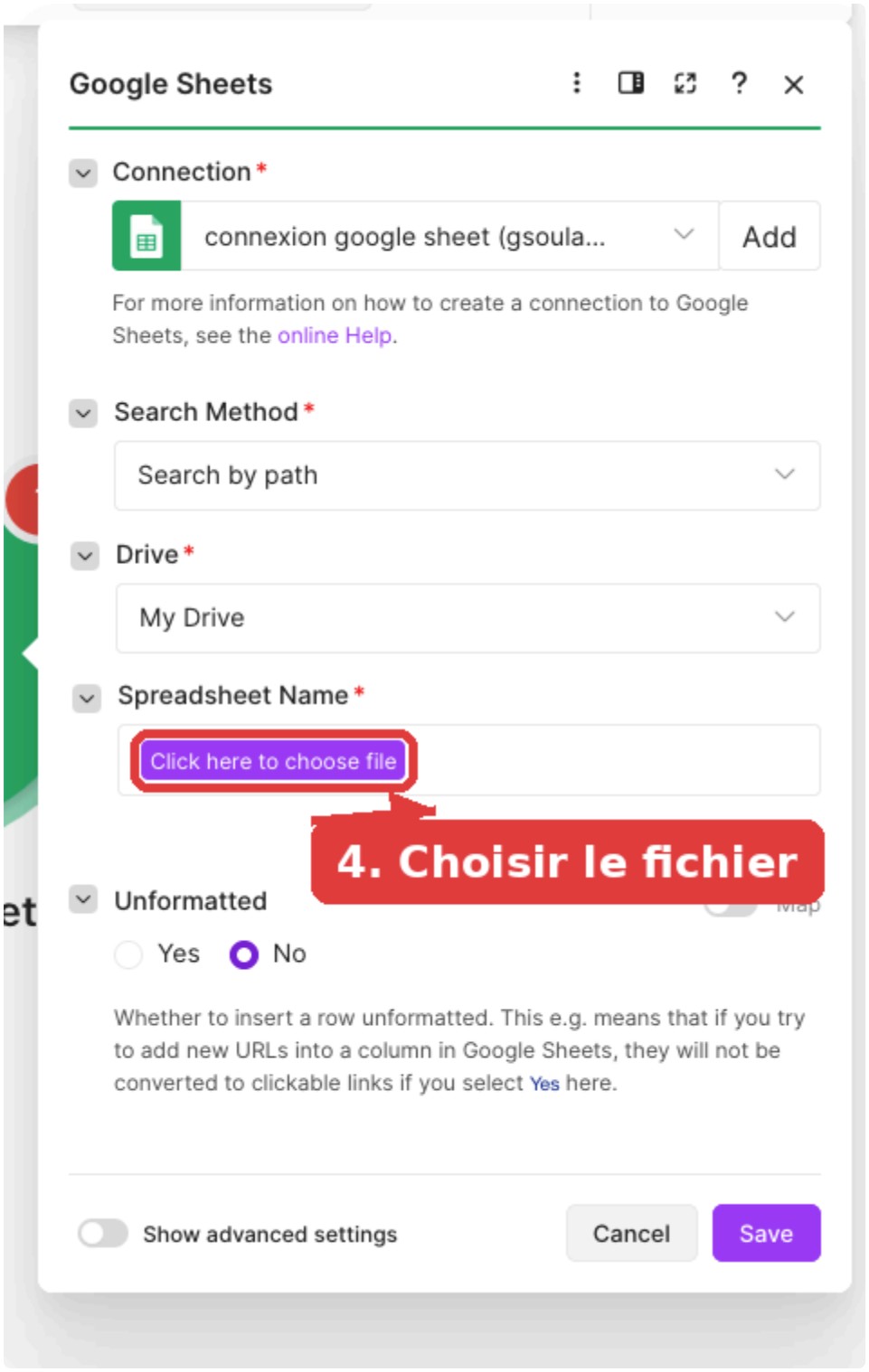
1. Cliquez sur la demi-bulle **+** à droite du module RSS, puis sur **Add module**.
2. Tapez `google` dans la recherche. Sous **Google Sheets**, cliquez sur **Add a Row** (« Appends a new row to the bottom of the table » : ajoute une ligne en bas du tableau).





La fenêtre **Google Sheets** s'ouvre. Remplissez les champs dans l'ordre : chaque choix fait apparaître le champ suivant.

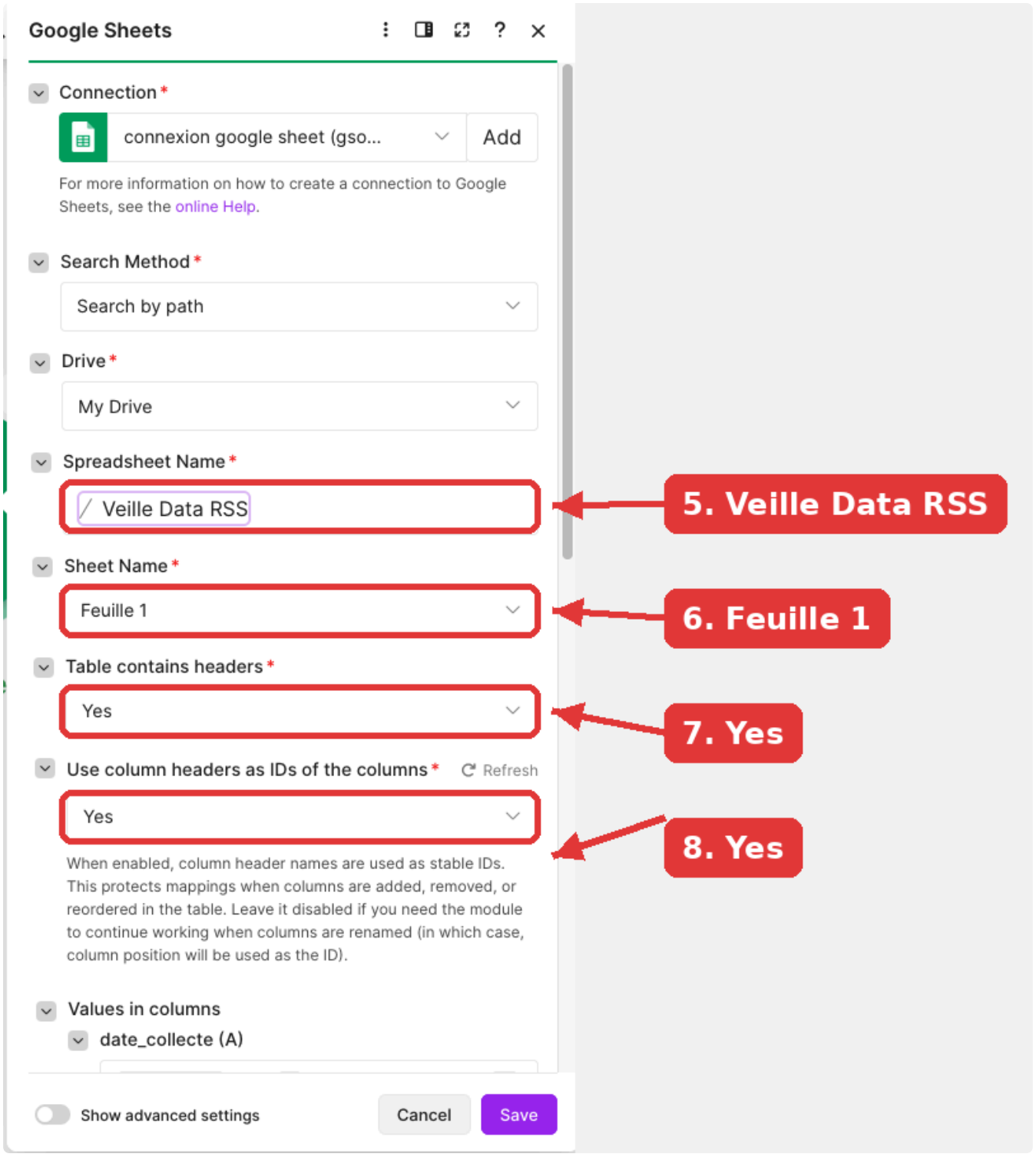
Champ	Ce que vous choisissez	Pourquoi
Connection	Votre connexion Google Sheets (créée avec le premier tuto)	C'est elle qui autorise Make à écrire dans votre Drive
Search Method	`Search by path`	Pour choisir le fichier en naviguant dans vos dossiers, sans copier d'identifiant
Drive	`My Drive`	Le Sheet est dans votre propre Drive, pas dans un Drive partagé
Spreadsheet Name	Cliquez sur **Click here to choose file** et sélectionnez Veille Data RSS	Le fichier où les lignes seront ajoutées
Unformatted	Laissez `No`	Avec No, Google Sheets transforme les URL en liens cliquables



Une fois le fichier choisi, trois nouveaux champs apparaissent :

Champ	Ce que vous choisissez	Pourquoi
Sheet Name	L'onglet où sont vos en-têtes (souvent `Feuille 1`)	Un fichier peut avoir plusieurs onglets : on indique lequel remplir

Champ	Ce que vous choisissez	Pourquoi
Table contains headers	`Yes`	La ligne 1 contient des en-têtes : Make écrira à partir de la ligne 2 et affichera le nom de chaque colonne
Use column headers as IDs of the columns	`Yes`	Make repère les colonnes par leur nom et non par leur lettre : si vous insérez une colonne plus tard, le mapping ne se décale pas



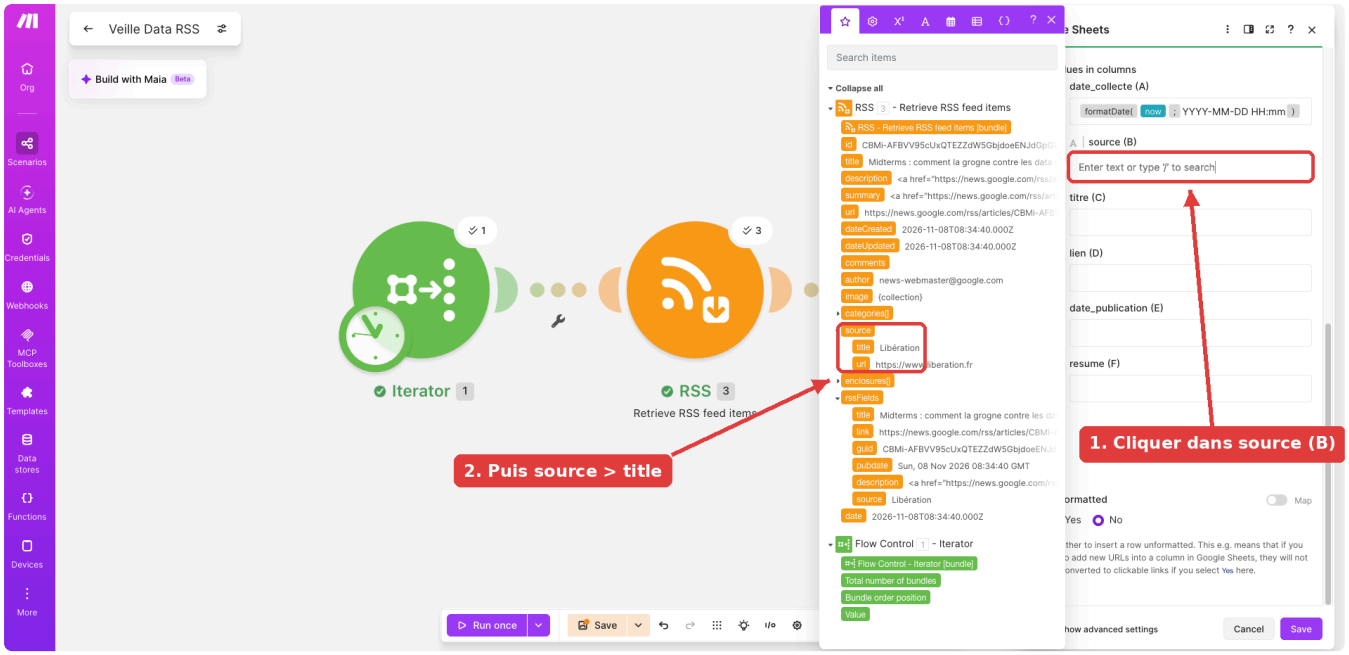
Ce qu'il faut mettre dans chaque colonne

Sous **Values in columns**, Make affiche une case par en-tête du Sheet. Cliquez dans chaque case : le panneau violet s'ouvre avec les données des modules précédents. Les formules ci-dessous supposent que le module RSS porte le numéro **3** (le petit chiffre gris à côté de « RSS ») : si le vôtre a un autre numéro, cliquez plutôt sur le champ dans le panneau.


Google Sheets
Values in columns
date_collecte (A)
source (B)
titre (C)
lien (D)
date_publication (E)
resume (F)
6. Une case par colonne
Unformatted
Map
Yes
No
Whether to insert a row unformatted. This e.g. means that if you try to add new URLs into a column in Google Sheets, they will not be converted to clickable links if you select Yes here.
Show advanced settings
Cancel
Save

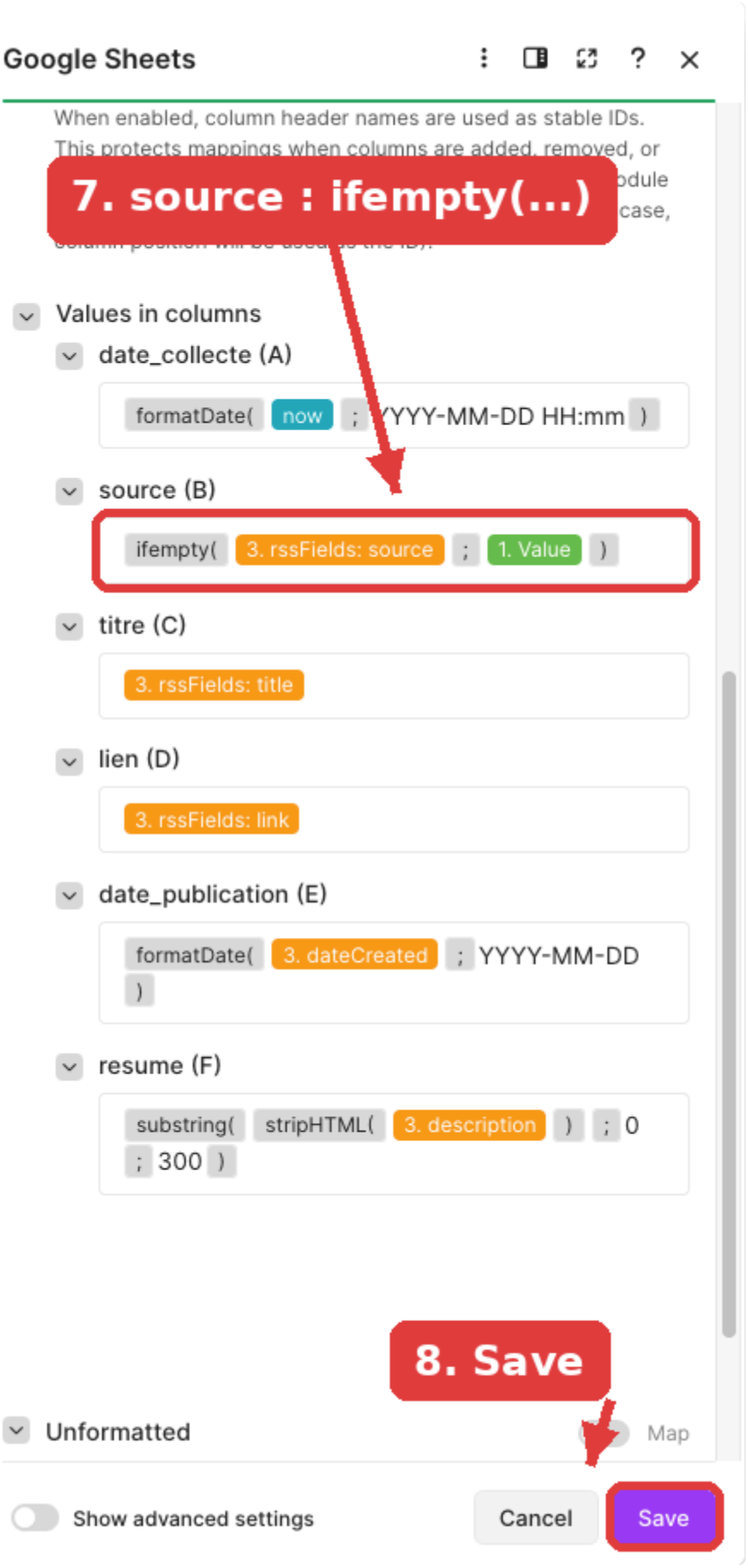
Colonne	Ce que vous tapez ou cliquez	Ce que le champ doit afficher	Pourquoi
date_collecte (A)	Collez `{{formatDate(now; "YYYY-MM-DD HH:mm")}}`	`formatDate( now ; YYYY-MM-DD HH:mm )`	Savoir quand l'article a été récupéré, dans un format triable
source (B)	Collez `{{ifempty(3.rssFields.source; 1.value)}}`	`ifempty( 3. rssFields: source ; 1. Value )`	Le nom du média (ex. Libération), fourni par Google News. Les autres flux ne le donnent pas : `ifempty` met alors l'URL du flux à la place, comme `source or url` en Python
titre (C)	Panneau : sous **RSS 3 > rssFields**, cliquez **title**	Bloc orange `3. rssFields: title`	Le titre de l'article
lien (D)	Panneau : sous **RSS 3 > rssFields**, cliquez **link**	Bloc orange `3. rssFields: link`	Le lien pour lire l'article complet
date_publication (E)	Collez `{{formatDate(3.dateCreated; "YYYY-MM-DD")}}`	`formatDate( 3. dateCreated ; YYYY-MM-DD )`	La date de publication, au format AAAA-MM-JJ pour pouvoir trier et filtrer
resume (F)	Collez `{{substring(stripHTML(3.description); 0; 300)}}`	`substring( stripHTML( 3. description ) ; 0 ; 300 )`	`stripHTML` retire les balises HTML de Google News, `substring` garde les 300 premiers caractères

Lire le panneau des données RSS. Après le Run once de l'étape 4, le panneau montre pour chaque champ un exemple de valeur du premier article : `title`, `description`, `url`, `dateCreated` … Le champ `source` est un petit dossier (une collection, comme un dictionnaire imbriqué en Python : `article["source"]["title"]`) qui contient le nom du média et son site.



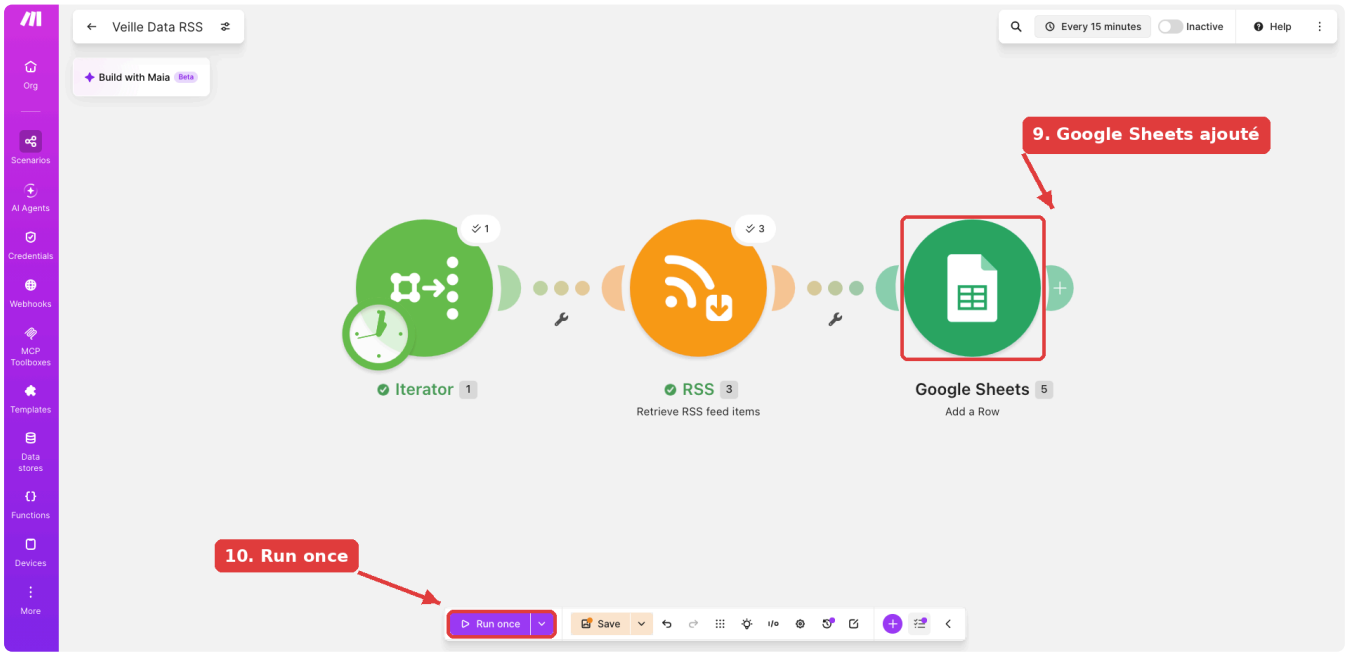
`title` ou `rssFields: title` ? Dans le panneau, les mêmes informations apparaissent deux fois : en haut (`title`, `url`, `source`…) et dans le dossier `rssFields` (`title`, `link`, `source`…). Les champs du haut sont normalisés par Make : ils ont le même nom quel que soit le flux. `rssFields` contient les balises brutes du flux, qui varient d'un site à l'autre. Les deux fonctionnent pour le titre et le lien. Pour la source, attention : seul Google News fournit le nom du média, les autres flux laisseront la case vide, d'où le `ifempty` du tableau.

Résultat attendu, avant de cliquer sur **Save** :


Google Sheets
When enabled, column header names are used as stable IDs.
This protects mappings when columns are added, removed, or
odule
case,
7. source : ifempty(...)
Values in columns
date_collecte (A)
formatDate(now ; YYYY-MM-DD HH:mm)
source (B)
ifempty(3. rssFields: source ; 1. Value)
titre (C)
3. rssFields: title
lien (D)
3. rssFields: link
date_publication (E)
formatDate(3. dateCreated ; YYYY-MM-DD)
resume (F)
substring(stripHTML(3. description) ; 0 ; 300)
8. Save
Unformatted
Map
Show advanced settings
Cancel
Save

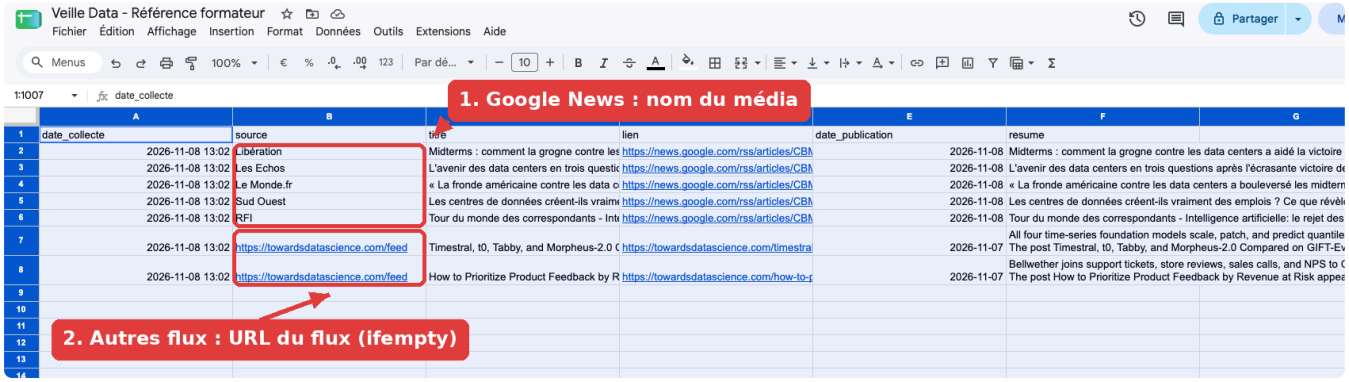
Après **Save**, le scénario compte trois modules reliés : **Iterator 1 → RSS 3 → Google Sheets 5**. Les numéros ne se suivent pas : Make numérote les modules dans l'ordre de création, y compris ceux que vous avez supprimés en route. Ce n'est pas grave, mais c'est pour ça que les formules doivent utiliser **votre** numéro de module RSS.

Les bulles de l'Iterator et du RSS datent du test précédent ; Google Sheets n'en a pas encore, car il n'a jamais tourné. Cliquez sur **Run once** pour tester la chaîne complète.



Vérifier le résultat dans le Sheet

Ouvrez votre Google Sheet : une ligne par article est apparue sous les en-têtes.



Ce que vous voyez	Pourquoi
5 lignes avec un nom de média en source (Libération, Les Echos…)	Google News fournit le nom du média, `ifempty` le garde
2 lignes avec l'URL du flux en source	Towards Data Science ne fournit pas de source : `ifempty` a pris l'URL du flux

Ce que vous voyez	Pourquoi
Aucune ligne SQLBI	Normal si SQLBI n'a rien publié dans les 7 derniers jours : c'est le rôle de Date from
date_publication au format 2026-11-08	`formatDate` a bien converti la date de l'article
Les liens sont cliquables	Unformatted est resté sur No

Point d'attention pour l'étape 6. Regardez les titres Google News : plusieurs parlent de « data centers » sous l'angle politique ou économique, pas d'analyse de données. Ils contiennent pourtant le mot « data ». C'est exactement le genre de bruit que le filtre de l'étape 6 devra traiter : il faudra choisir des mots-clés plus précis que « data » seul.

En Python, ce mapping équivaut à :

```
ligne = {
    "date_collecte": maintenant.strftime("%Y-%m-%d %H:%M"),
    "source": article["rssFields"]["source"] or url,  # ifempty(...)
    "titre": article["title"],
    "lien": article["url"],
    "date_publication": article["dateCreated"].strftime("%Y-%m-%d"),
    "resume": supprimer_html(article["description"])[:300],
}
```

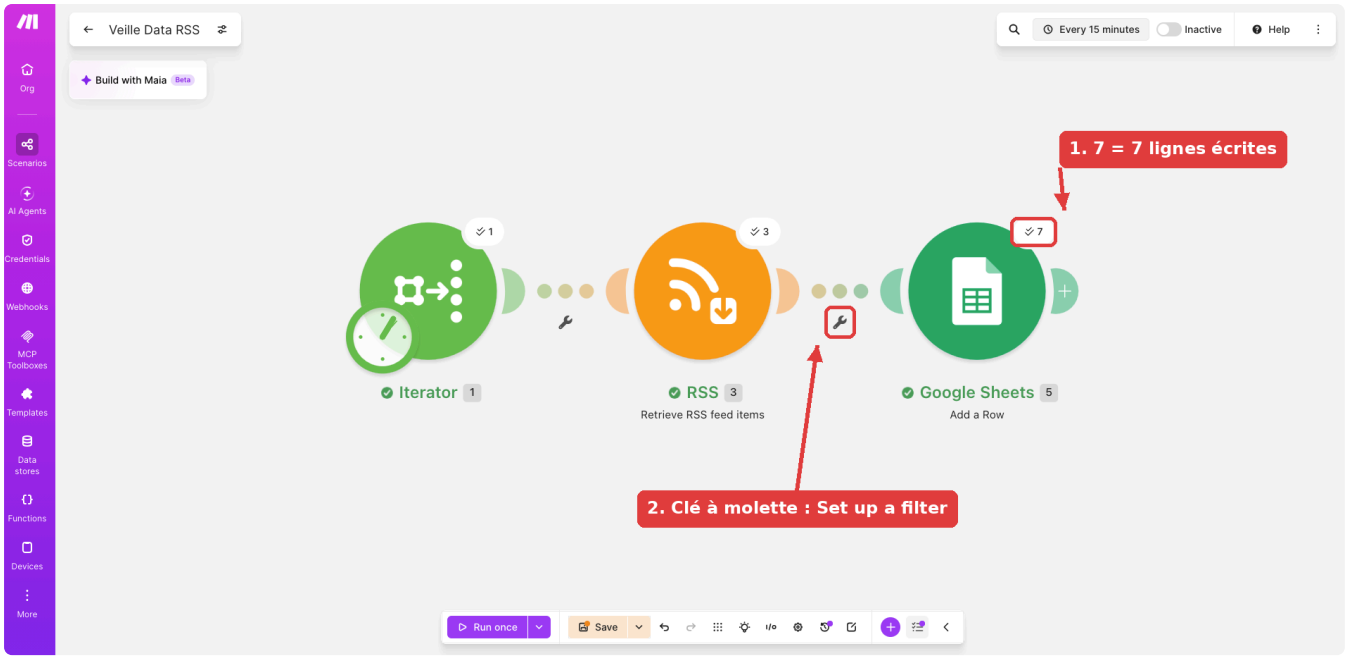
Sauvegardez : cliquez sur **Save** dans la barre du bas.

Étape 6 — Filtrer les articles par mots-clés

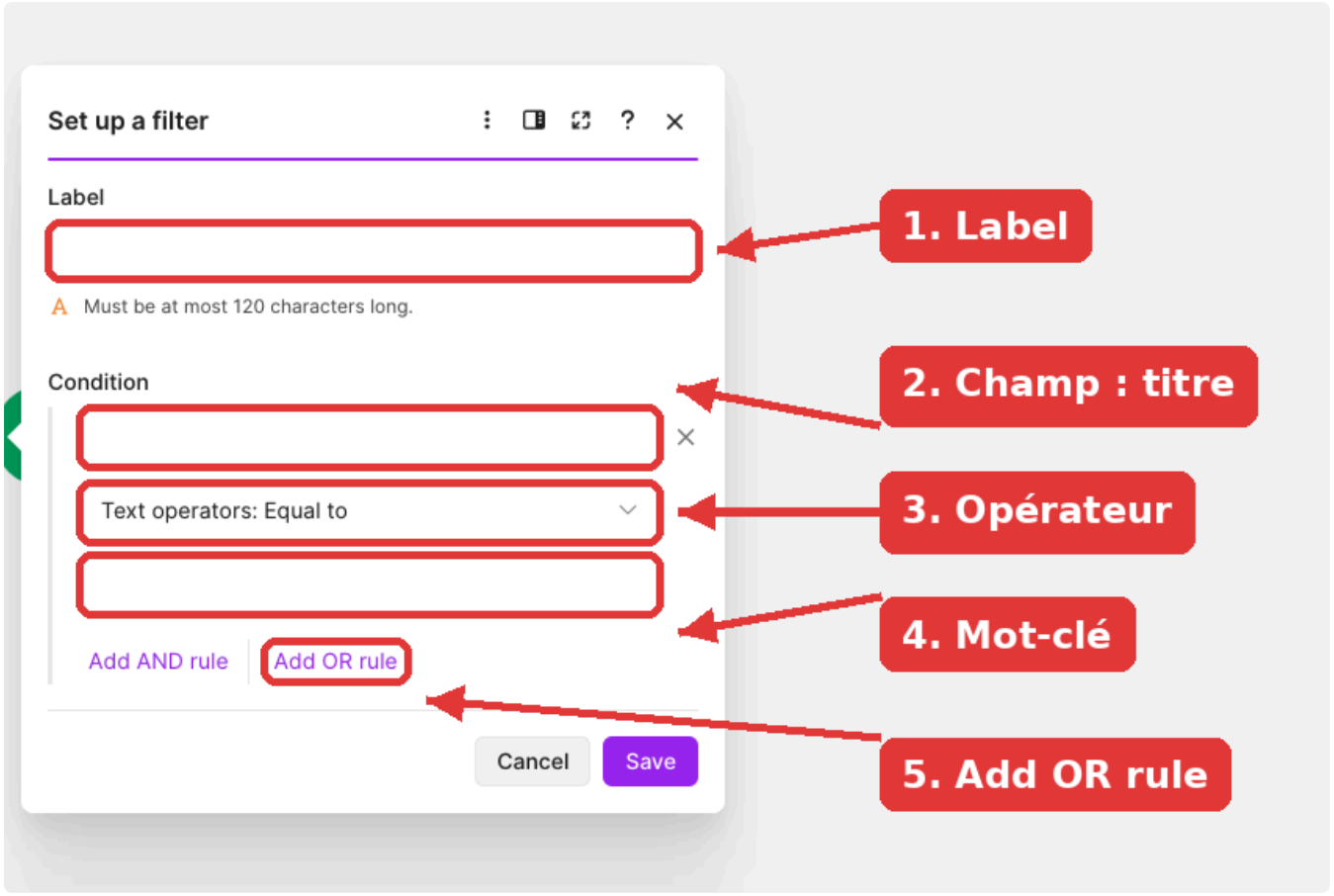
Pourquoi un filtre : le test de l'étape 5 a écrit tous les articles récents, y compris ceux qui parlent de « data centers » sous l'angle politique. Le filtre se place **sur la liaison** entre RSS et Google Sheets : seuls les articles qui passent la condition continuent vers le Sheet. En Python, c'est le `if parle_de_data(article):` avant `ajouter_ligne(...)`.

Lire les bulles après le Run once complet. Iterator **1**, RSS **3**, Google Sheets **7** : le module Google Sheets a tourné 7 fois, une fois par article, donc 7 lignes écrites.

1. Cliquez sur la **clé à molette** sous les points entre RSS et Google Sheets.
2. Choisissez **Set up a filter**.



La fenêtre **Set up a filter** s'ouvre. Une condition se lit comme une phrase : *[champ] [opérateur] [valeur]*, par exemple « le titre contient Power BI ».



Ce qu'il faut mettre dans chaque champ du filtre

Repère	Champ	Ce que vous tapez ou choisissez	Pourquoi
1	Label	`Mots-clés data`	Le nom s'affiche sur la liaison : on voit d'un coup d'œil ce que fait le filtre
2	Premier champ de la condition	Cliquez dedans, puis dans le panneau sous **RSS 3 > rssFields**, cliquez **title**	On filtre sur le titre de l'article
3	Opérateur	Ouvrez la liste et choisissez **Text operators: Contains (case insensitive)**	« Contient » plutôt que « égal à » : le mot peut être n'importe où dans le titre. Case insensitive : `Power BI`, `power bi` et `POWER BI` passent tous
4	Valeur	Tapez le premier mot-clé : `analyse`	Le mot cherché dans le titre
5	Add OR rule	Cliquez pour ajouter une nouvelle condition, et refaites 2, 3 et 4 avec le mot-clé suivant	Chaque mot-clé est une condition. OR = il suffit qu'une seule soit vraie pour que l'article passe

Après la première règle et un clic sur **Add OR rule**, une deuxième règle vide apparaît sous un séparateur **or**. Elle repart avec l'opérateur par défaut « Equal to » : il faut re-choisir le champ et « Contains (case insensitive) » à chaque nouvelle règle.



Mots-clés à mettre, une règle OR par mot. Pourquoi deux listes : vos flux mélangent le français (Google News) et l'anglais (Towards Data Science, SQLBI). Un mot-clé français ne trouvera jamais un titre anglais : `analyse` ne correspond pas à « analysis ».

Langue	Mots-clés	Pour attraper
Français	`analyse`, `analyse de données`, `data analyst`, `tableau de bord`	Les articles Google News
Anglais	`analysis`, `analytics`, `dashboard`, `visualization`, `KPI`	Towards Data Science, Storytelling with Data
Outils (toutes langues)	`Power BI`, `DAX`, `SQL`, `Python`	SQLBI et les tutoriels techniques

Pourquoi pas `data` ni `données` seuls : ils laissent passer les articles sur les « data centers » et les « centres de données », qui parlent d'énergie ou d'emploi, pas d'analyse.

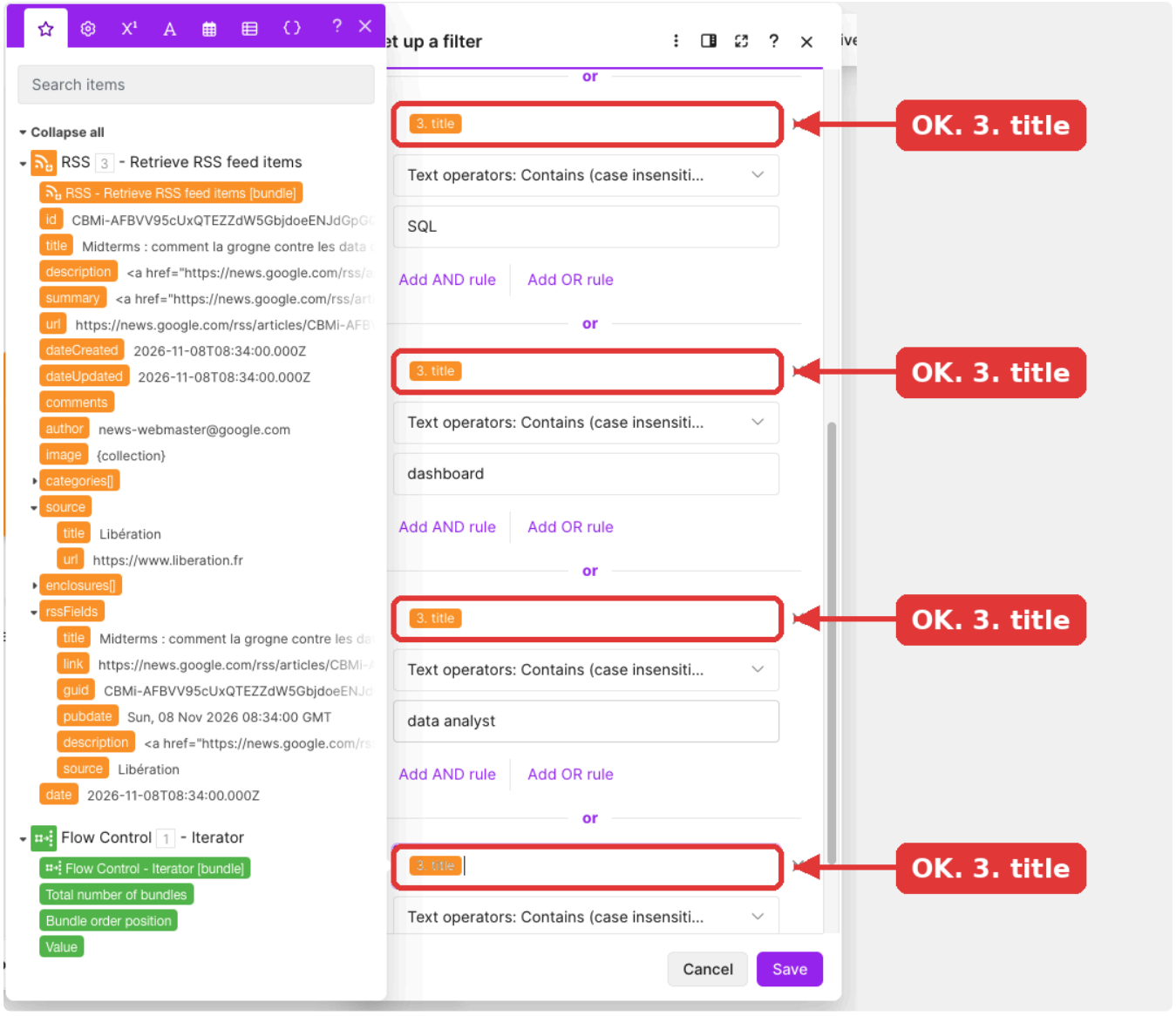
Avant de choisir vos mots-clés, regardez les vrais titres de vos flux : cliquez sur la bulle du module RSS après un Run once et lisez les `title` de chaque Operation. C'est le réflexe de l'analyste : partir des données réelles, pas de ce qu'on imagine.

AND ou OR ? Add OR rule crée une alternative : l'article passe si le titre contient `analyse` **ou** `SQL` **ou**… **Add AND rule** ajoute une condition supplémentaire à remplir en même temps : avec AND, le titre devrait contenir tous les mots à la fois, et presque aucun article ne passerait. En Python :

```
mots_cles = [
    "analyse", "analyse de données", "data analyst", "tableau de bord",   #
français
    "analysis", "analytics", "dashboard", "visualization", "kpi",       #
anglais
    "power bi", "dax", "sql", "python",                                  #
outils
]
titre = article["rssFields"]["title"].lower()          # case insensitive

if any(mot in titre for mot in mots_cles):              # OR entre les règles
    ajouter_ligne(sheet, article)
```

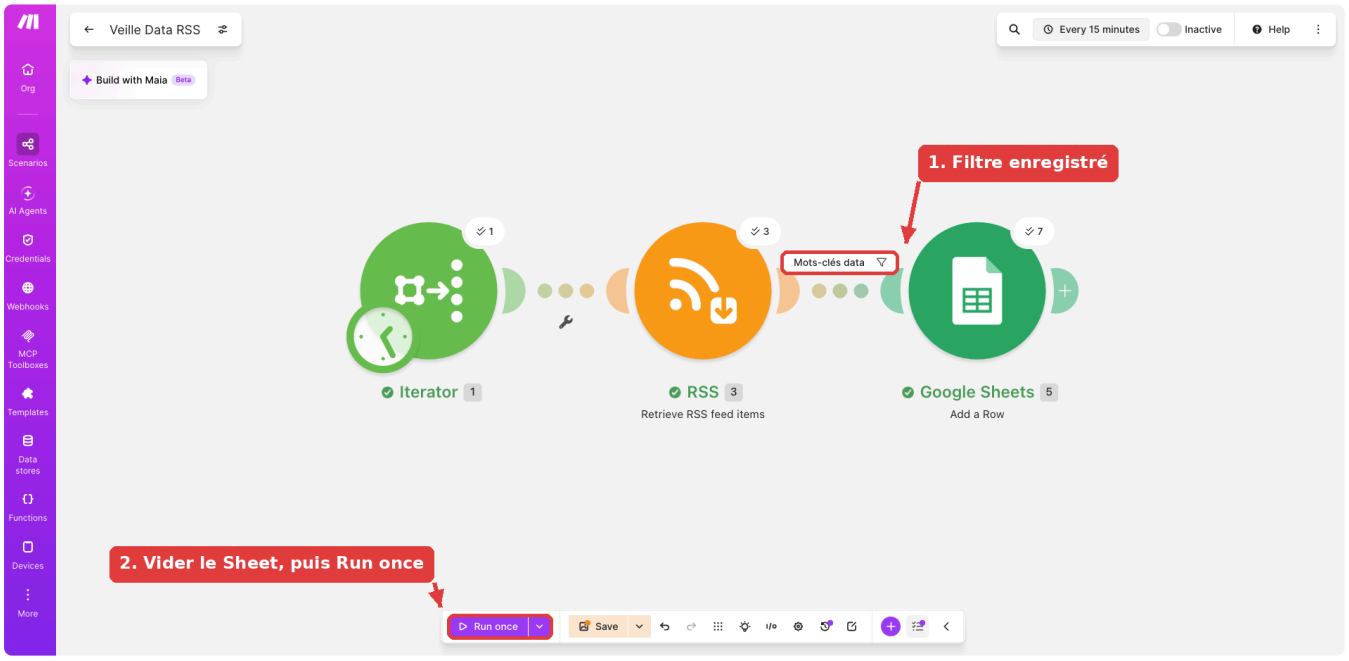
Chaque règle commence par le bloc orange `3. title`, puis « Contains (case insensitive) », puis le mot-clé tapé en texte dans la troisième case. Faites défiler toute la fenêtre pour vérifier toutes les règles avant de cliquer sur **Save**.



Après **Save**, le nom du filtre « Mots-clés data » et une icône d'entonnoir s'affichent sur la liaison entre RSS et Google Sheets : c'est le signe que le filtre est en place. Les bulles (1, 3, 7) datent encore de l'exécution précédente.

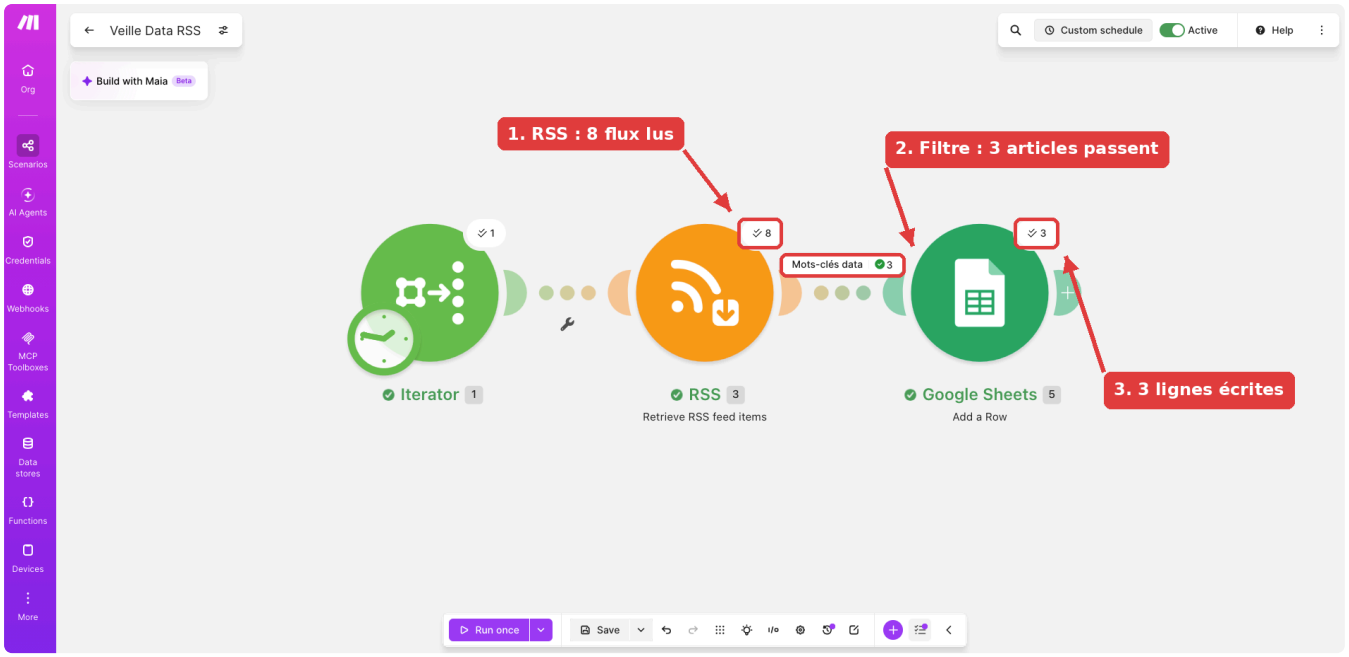
Pour tester le filtre :

1. Ouvrez le Google Sheet et supprimez les lignes de test, en gardant la ligne 1 des en-têtes. Pourquoi : on veut voir uniquement ce que le filtre laisse passer.
2. Revenez dans Make et cliquez sur **Run once**.

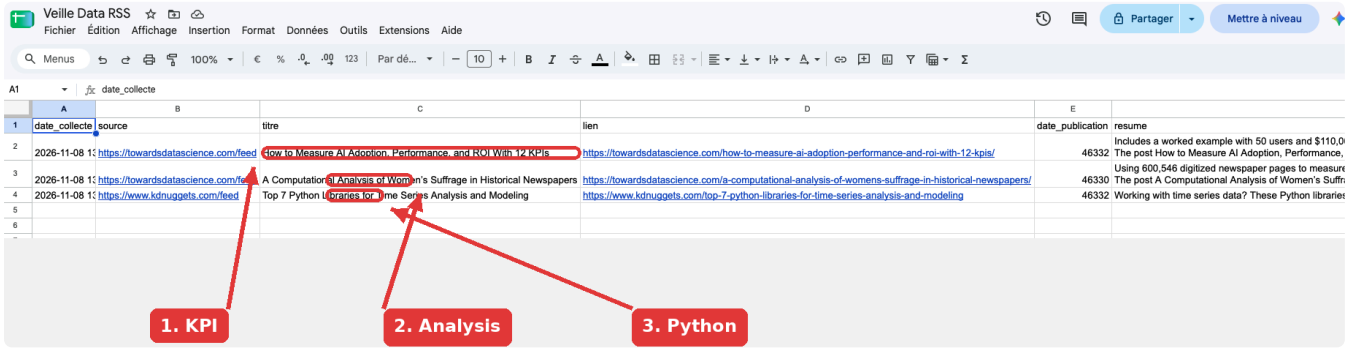


Résultat attendu

Après le Run once, la bulle RSS affiche **8** (un passage par flux). Le filtre affiche lui aussi un compteur : ici **3** articles ont passé les mots-clés. Google Sheets affiche **3** : trois lignes écrites. Opérations consommées : 1 + 8 + 3 = 12. Le nombre d'articles retenus varie d'une semaine à l'autre, selon ce que publient les flux.



Dans le Sheet **Veille Data RSS**, chaque ligne correspond à un article retenu par le filtre. Ici : le premier grâce à **KPI** (« 12 KPIs »), le deuxième grâce à **analysis** (« Computational Analysis »), le troisième grâce à **Python** (« Top 7 Python Libraries »). Trois titres anglais, venus de Towards Data Science et KDnuggets : d'où l'intérêt des mots-clés dans les deux langues.



Sauvegardez : cliquez sur **Save** dans la barre du bas pour enregistrer le filtre dans le scénario.

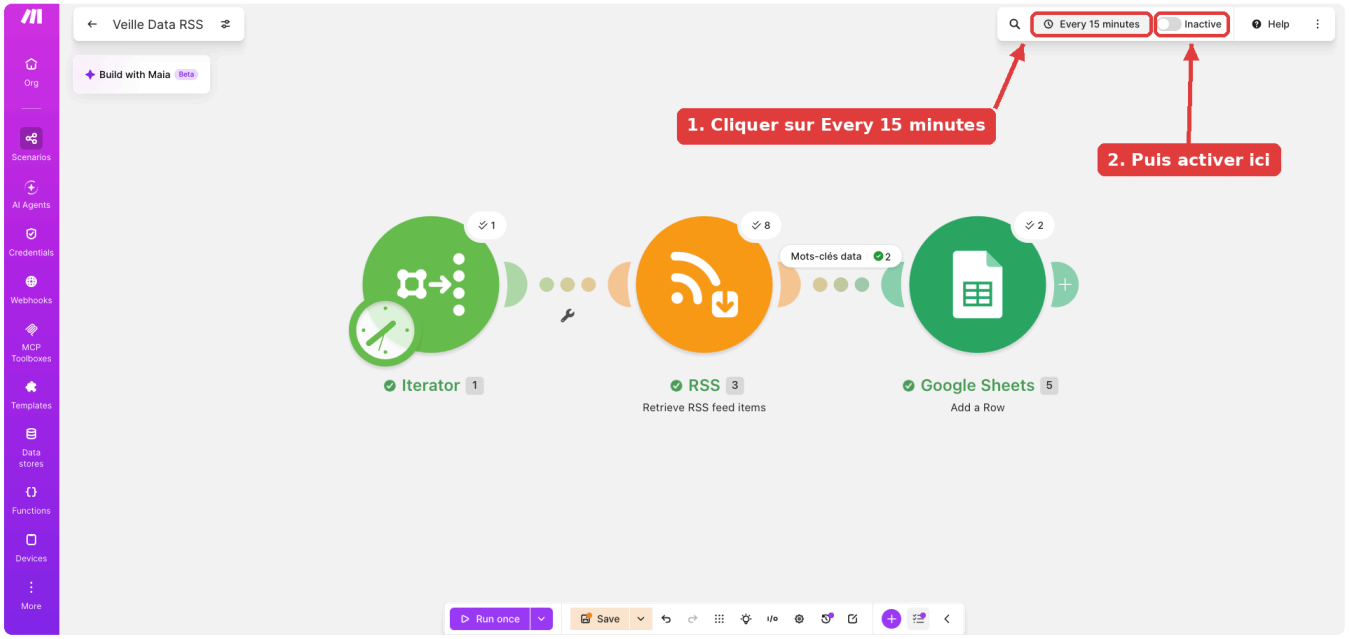
Étape 7 — Planifier une fois par semaine et activer

Pourquoi : jusqu'ici, le scénario ne tourne que quand vous cliquez sur Run once. La planification le fait tourner tout seul, chaque lundi matin, juste avant le point veille de l'équipe. C'est ce qui transforme le scénario en veille **automatique**.

Avant d'activer, ouvrez le Google Sheet et supprimez les lignes de test, en gardant la ligne 1 des en-têtes. Pourquoi : la première exécution automatique relira les 7 derniers jours et réécrirait ces articles en doublon.

En haut à droite de l'éditeur, deux boutons pilotent l'exécution automatique :

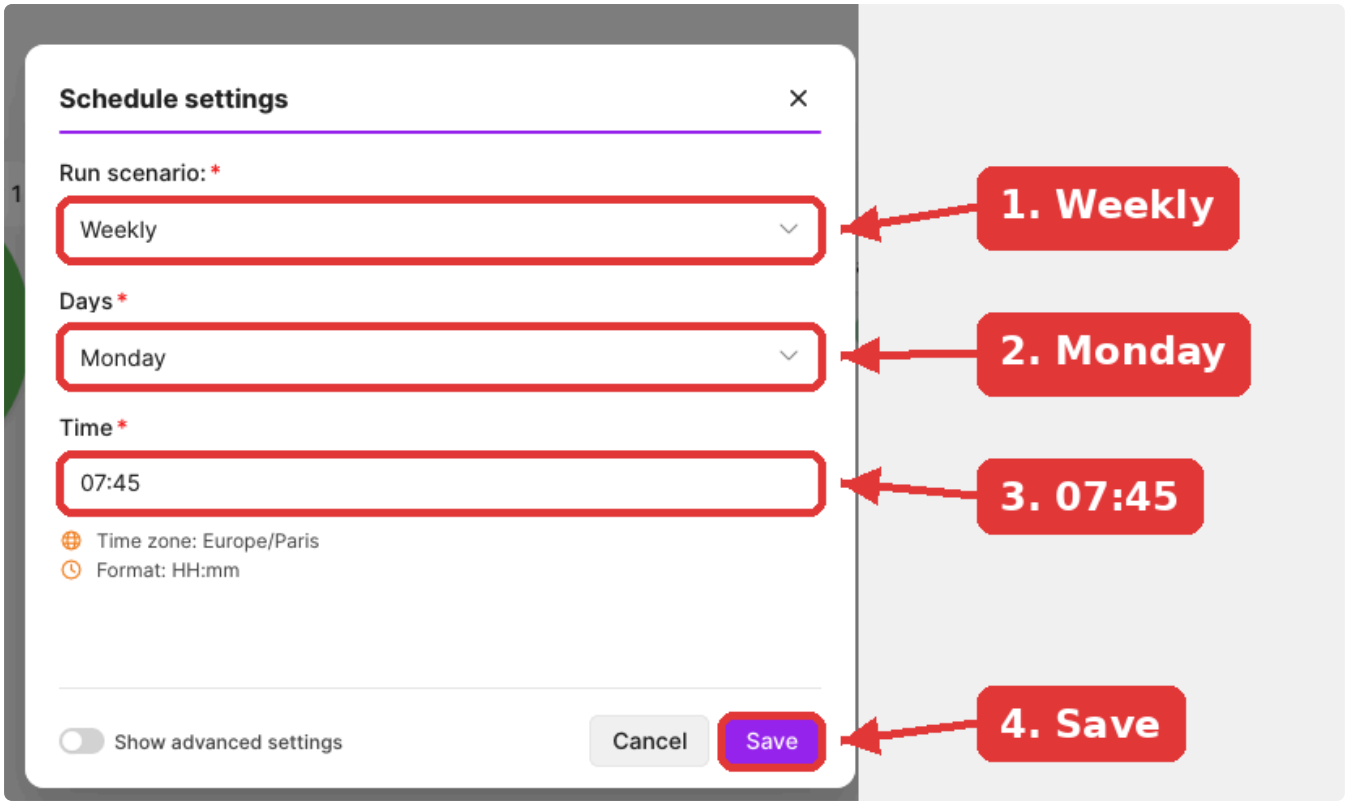
1. **Every 15 minutes** : la fréquence actuelle (réglage par défaut de Make). Cliquez dessus pour la changer.
2. **Inactive** : l'interrupteur qui allume ou éteint la planification. Tant qu'il est sur Inactive, le scénario ne tourne jamais tout seul.



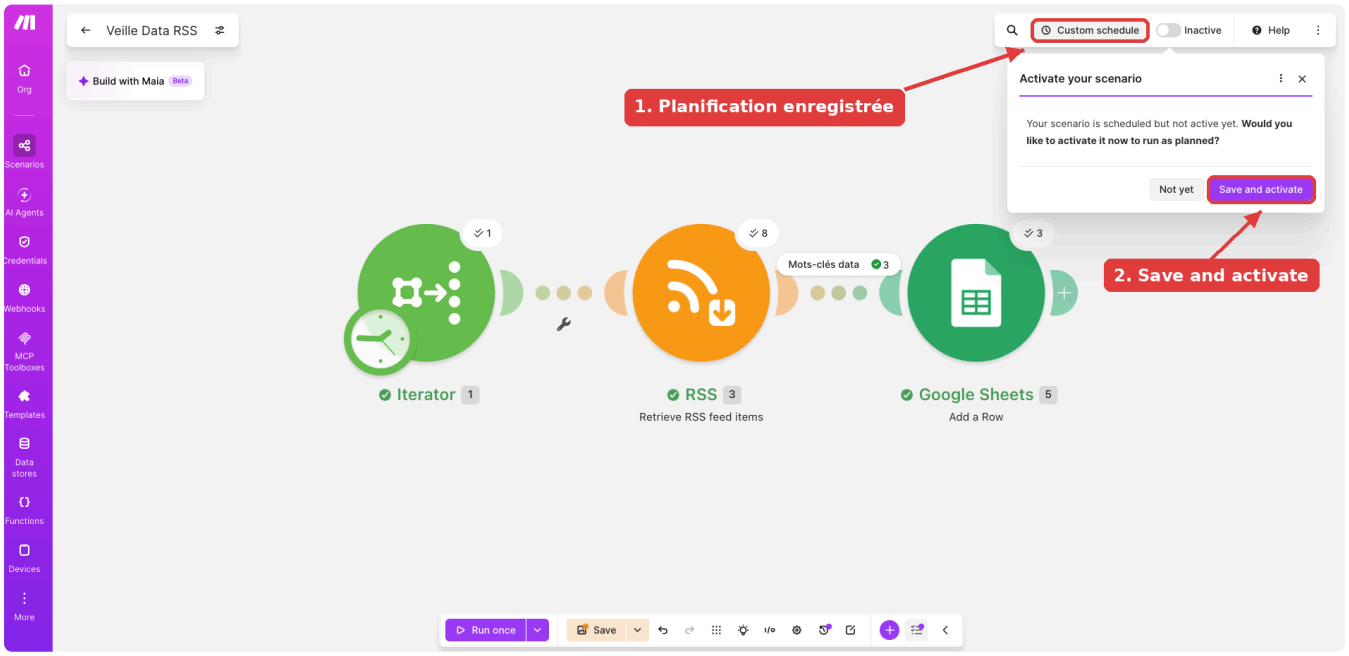
Ce qu'il faut choisir dans la fenêtre de planification

La fenêtre **Schedule settings** s'ouvre :

Repère	Champ	Ce que vous choisissez	Pourquoi
1	Run scenario	`Weekly`	Une exécution par semaine
2	Days	`Monday`	Le point veille de l'équipe a lieu le lundi
3	Time	Tapez `07:45` (format HH:mm)	Les articles sont dans le Sheet avant l'arrivée de l'équipe. L'heure est celle de Paris (Time zone: Europe/Paris)
4	Save	Cliquez	Enregistre la planification

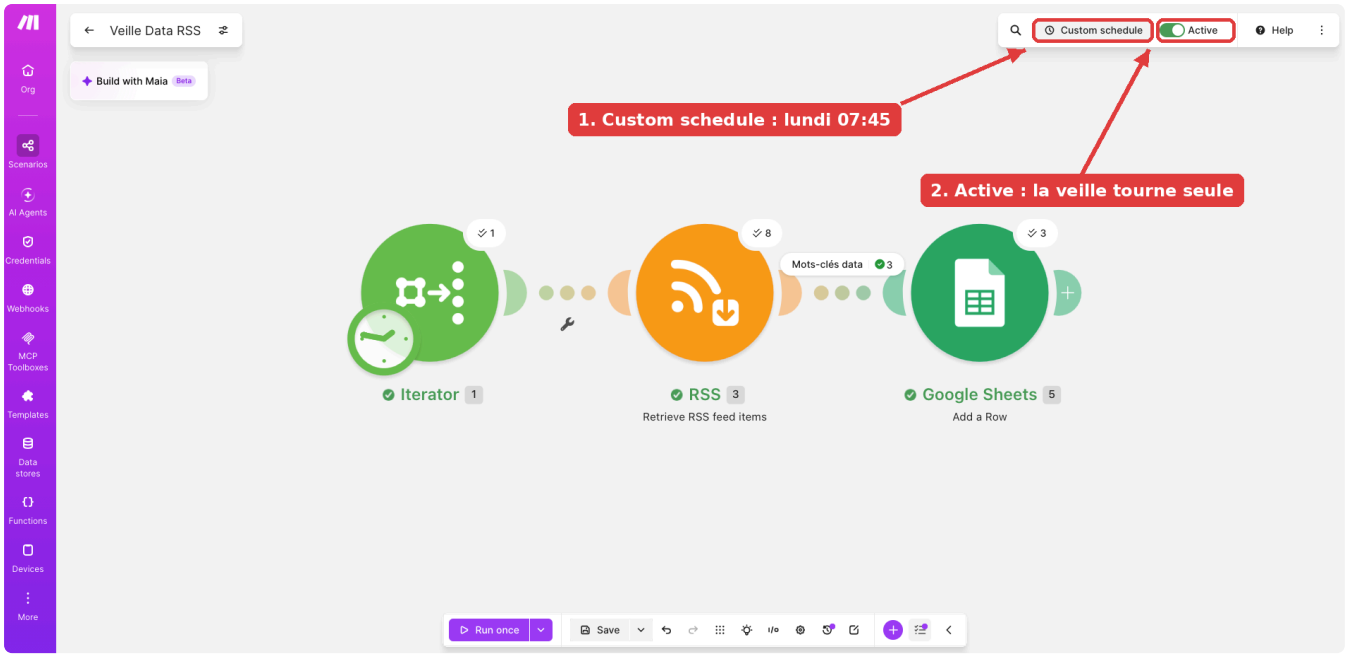


Après **Save**, le bouton du haut affiche **Custom schedule** à la place de « Every 15 minutes » : la planification est enregistrée. Make ouvre aussitôt une fenêtre **Activate your scenario** : le scénario est planifié mais pas encore actif. Cliquez sur **Save and activate**. Pourquoi : planifier dit *quand* le scénario doit tourner, activer l'autorise à tourner. Sans activation, la planification ne se déclenche jamais.



Si Make refuse l'activation, c'est que vous avez déjà 2 scénarios actifs, le maximum de l'offre gratuite : désactivez-en un autre depuis la liste **Scenarios**, puis recommencez.

Le scénario est actif : l'interrupteur est vert sur **Active** et le bouton affiche **Custom schedule** (chaque lundi à 07:45).



C'est terminé

Votre veille tourne maintenant toute seule : chaque lundi à 7 h 45, Make lit les 8 flux, garde les articles de la semaine qui parlent d'analyse de données et les ajoute au Sheet **Veille Data RSS**.

Pour vérifier qu'elle a bien tourné, ouvrez le scénario le lundi et regardez l'historique des exécutions (icône d'horloge dans la barre du bas) : chaque exécution y apparaît avec sa date et son nombre d'opérations. Pour la mettre en pause, repassez l'interrupteur sur **Inactive**.

Pourquoi le lundi à 7 h 45 et Date from à -7 vont ensemble : chaque lundi, le module RSS lit les articles publiés depuis le lundi précédent. Aucun jour n'est oublié, aucun article n'est lu deux fois. En Python, c'est l'équivalent d'une tâche planifiée (un `cron` du type `45 7 * * 1`) qui lance le script chaque lundi.